

National Emission Standards for Hazardous Air Pollutants (NESHAP) Maximum Achievable Control Technology (MACT) Floor Analysis for Example Industry Units for Final Rule

OAR-OCAP-NRD-EDAB and/or author names here

November 20, 2025

Introduction

Section 112 of the Clean Air Act (CAA) requires the the U.S. Environmental Protection Agency (EPA) establish National Emission Standards for Hazardous Air Pollutants (NESHAP) for the control of the hazardous air pollutants (HAP) emitted from both new and existing sources in a source category. These standards must reflect the maximum degree of reduction in emissions that is achievable. The minimum level of control is referred to as the “Maximum Achievable Control Technology (MACT) floor.” The method for determining the MACT floor as the minimum control level allowed for establishing standards for a NESHAP is defined for both new and existing sources by CAA Section 112(d)(3). For new sources, the MACT floor cannot be less stringent than the emission control that is achieved in practice by the best-controlled similar source. For existing sources, the MACT floor cannot be less stringent than the average emission limitation achieved by the best-performing 12 % of existing sources for source categories with 30 or more sources or those governed by CAA Section 129(a)(2), or the best-performing 5 sources for source categories with fewer than 30 sources for those governed by CAA Section 112(d)(3).

The purpose of this memorandum is to present the methodology and the results of the MACT floor analysis for the NESHAP source category Example Industry. This MACT floor analysis uses data collected in a nationwide survey of Example Industry unit owners and operators conducted by the EPA in 2025 under the Information Collection Request for NESHAP for Example Industry.

MACT Floor Analysis Hazardous Air Pollutants

The specific chemicals, compounds, or groups of compounds designated to be HAP are listed in *CAA section 112(b)*. From this HAP list, the following HAP, among others, were identified to be emitted from Example Industry units and were included in the MACT floor analysis: Chemical X, Chemical X, Chemical X, Chemical Y.

MACT Floor Analysis subcategories

Under *CAA section 112(d)(1)*, the EPA has the discretion to “...distinguish among classes, types, and sizes of sources within a category or subcategory in establishing...” standards. When separate subcategories are established, a MACT floor is determined separately for each subcategory. To determine whether the EGU source category warrants subcategorization for the MACT floor analysis, the EPA reviewed Example Industry design, operating information, and air emissions data compiled in the ICR data set and other information collected by the Agency for development of the NESHAP for this source category. Based on this review, the EPA concluded that there are significant design and operational differences in Example Industry that affect the levels of emissions of certain HAP to justify subcategorizing Example Industry for the purpose of establishing MACT floors.

For the MACT floor analysis, the following Example Industry subcategories were defined:

- **HAP Chemical X: Subcategory, Size:** *describe it here and how/why it was determined*
Facility size was determined as greater or less than ... and indicated by ‘A’ for the first set and ‘B’ for the second set. Since subcategory ‘B’ is a new designation, both existing and new source MACT floor analyses were needed. For *facility size* ‘A’ only existing source MACT floor analyses were calculated.
- **HAP Chemical Y: No subcategory** *describe it here and how/why it was determined*
 The facility size subcategory does not affect HAP Chemical Y, no subcategory differentiation was applied.

Calculating Emission Values used in the MACT Floor Analysis

The EPA received emissions data from industry respondents under the Example Industry MACT ICR in a number of different units of measure and converted these units of measure to a common ppm, ppm, ppm, mg/m³/s for Chemical X, Chemical X, Chemical X, Chemical Y respectively.

If any outlier analysis or subsequent QA/QC was done on the emissions data prior to MACT floor analysis, please include it here.

MACT Floor Analysis Methodology

The first step in the MACT floor analysis for each subcategory and HAP (or surrogate) was to rank all of the Example Industry units in the subcategory (for which there were emissions data in the ICR data set) for the pollutant type by emission level (lowest to highest). From this ranking, a MACT floor pool of sources was identified for determining the minimum control level allowed for the MACT floor, consistent with the criteria defined for new and existing sources by CAA Section 112(d)(3). For the new-source MACT floors, the best-controlled similar source was identified for which there were individual source test run data in the ICR data set. The selection of top performing sources for the existing source MACT floor was performed using the `MACT_existing()` function in the `UPLforOAR` R package version 0.1.0, with details in Appendix B. The selection of the best performing source for the new source MACT floor was performed using the `MACT_new()` function in the `UPLforOAR` R package version 0.1.0, with details in Appendix B.

For the existing-source MACT floors, selection of the MACT floor pool size (i.e., number of units to be included in the determination of the average emission limitation value) was determined on an individual subcategory basis as described below.

- **Subcategory 1.** *do any subcategories need different handling?*
- **Subcategory 2.** *do any subcategories need different handling?*

Probability distribution Selection

The next step in the MACT floor analysis was to incorporate data variability into the calculations of the applicable MACT floor limits for the subcategories using an upper prediction limit (UPL) approach. Specifically, the UPL is calculated as the α -level significance, typically $\alpha = 0.99$, quantile of the average of future emission test values, typically three. The level of confidence represents the level of protection afforded to facilities whose emissions are in line with the best performers, and consequently, the level of confidence is not arbitrary. For example, a 99% level of confidence means that a facility whose emissions are in line with the best performers has one chance in 100 of exceeding the floor limit. A prediction interval for a single future observation (or an average of several test observations) is an interval that will, with a specified degree of confidence, contain the next (or the average of some other pre-specified number) of randomly selected observation(s) from a population. In other words, the UPL estimates what the upper bound of future values will be, based upon present or past background samples taken. The UPL consequently represents the value at which we can expect the mean of future observations for the HAP or HAP surrogate emissions to fall within a specified level of confidence, based upon the measurements from an independent sample from the same population. The UPL approach encompasses all the data point-to-data point variability.

Implementation

The form of the UPL equation differs depending upon the nature of the data set to which it is applied. To this end, the data sets were evaluated for each HAP and HAP surrogate to ascertain whether the data were normally, log-normally, skew-normally, beta, or gamma distributed. For each distribution, a probability density function was fit to the test run emissions using Bayesian Markov Chain Monte Carlo (MCMC). This analysis was implemented in R using `runjags` and JAGS (Just Another Gibbs Sampler) software, with three MCMC chains with different initial values, completely uninformed priors, a burn-in and adapt of 10,000 iterations each, and a final set of 10,000 iterations retained in posterior distributions. Random seeds were specified to control for stochasticity and ensure reproducibility. The JAGS model scripts were defined using the function `write_likelihood()`, and executed using functions `setup_likelihood()` and `run_likelihood()` in the UPLforOAR R package version 0.1.0. See Appendix B for more details.

Convergence was assessed quantitatively by checking Gelman-Rubin diagnostics and qualitatively by viewing plots of the iteration chains to ensure they were well-mixed and without trend, as well as histograms of the posterior distributions to ensure they converged. These checks were carried out using the functions `converge_likelihood()` and `converge_figs()` in the UPLforOAR R package version 0.1.0. See Appendix B for more details.

Both quantitative and qualitative evaluations were used to select the appropriate distribution. For each distribution, the Sum of Squared Errors (*SSE*) is calculated as:

$$SSE = \sum (y_i - \hat{y}_i)^2$$

Where y_i is the empirical test run probability density at emission value x_i and $\hat{y}_i$ is the predicted probability density at emission value x_i . The lower the *SSE*, the better the probability distribution represents the probability of the emission values. The empirical test run probability densities, y_i , were also checked if they were significantly different than the predicted probability densities, $\hat{y}_i$, by comparing them to the 95% confidence interval (CI) around the predicted densities derived from posterior distributions. The uncertainty around the predicted probability density can be variable; thus one can have a very different probability at the same x_i but be within the 95% CI if the uncertainty is very large, or have a significantly different probability at the same x_i even though the y_i and $\hat{y}_i$ are extremely similar if there is very little uncertainty. The more y_i that are statistically equivalent to $\hat{y}_i$, the better the distribution represents the probability of x_i emission values. A proper probability density function will integrate to 1, so if the integral of the fitted distribution is substantially different from 1 this can indicate an issue. Lastly, a visual assessment comparing the empirical and predicted probability densities across the range of possible emission values can identify any problematic behavior or differentiate between distributions. These fit metrics were calculated using the functions `fit_likelihood()` and `obs_density()` in the UPLforOAR R package version 0.1.0. See Appendix B for more details.

To calculate the UPL, which is typically the 99th quantile of the average of three future emission test values, the posterior distributions of three new draws from the fitted probability distribution were averaged and then the UPL is determined as the 99th quantile of this resulting distribution. The fitted distribution is truncated by lower and upper bounds of (typically zero and three times the maximum value) to prevent physically improbable emissions, which prevents extreme values from long-tailed distributions biasing the UPL and otherwise has no effect on the results. The 2.5th and 97.5th quantiles of the posterior distributions of the predicted probability density $\hat{y}_i$ at each x_i test run value were used to calculate 95% CI for the goodness-of-fit metrics. The median of the posterior distributions of the predicted probability density $\hat{y}_i$ were used in comparisons to y_i to calculate the *SSE*. The median of the posterior distributions for all $\hat{y}_i$ across the possible range of emission test run values was monitored for use in plotting the distributions and visual comparisons to the empirical distributions. All of these computations were done using the function `output_likelihood()` in the UPLforOAR R package version 0.1.0. See Appendix B for more details.

The parameter estimation and UPL calculations for each distribution investigated were performed using the function `Bayesian_UPL()` in the UPLforOAR R package version 0.1.0, which wraps the functions described above and concatenates the results for easy comparison. See Appendix B for more details.

UPL Calculation for Normal Data

In the case of normally distributed data, the MACT floor limit is an UPL calculated as the α -level quantile of the average of m new emission runs, where the probability density, y_i , of normally distributed emissions, x_i , is dependent on parameters μ , the mean, and σ the standard deviation:

$$y_i = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

UPL Calculation for Log-normal Data

In the case of log-normally distributed data, the MACT floor limit is an UPL calculated as the α -level quantile of the average of m new emission runs, where the probability density, y_i , of log-normally distributed emissions, x_i , is dependent on parameters μ , the mean of $\log(x_i)$ and σ , the standard deviation of $\log(x_i)$:

$$y_i = \frac{1}{x_i\sqrt{2\pi\sigma^2}} e^{-\frac{(\log(x_i) - \mu)^2}{2\sigma^2}}$$

UPL Calculation for Skew-normal Data

In the case of skew-normally distributed data, the MACT floor limit is an UPL calculated as the α -level quantile of the average of m new emission runs, where the skew-normal distribution is defined by three parameters; location ξ , scale ω , and skewness γ . This is equivalent to a normal distribution when $\gamma = 0$, it is right-skewed if $\gamma > 0$, and left-skewed if $\gamma < 0$. The probability density, y_i , of skewed distributed emissions is:

$$y_i = \frac{2}{\omega} \phi\left(\frac{x_i - \xi}{\omega}\right) \Phi\left(\gamma \left(\frac{x_i - \xi}{\omega}\right)\right)$$

Where $\phi(x)$ is the standard normal probability distribution, $\Phi(x)$ is its cumulative probability distribution function, and x_i is transformed to $\frac{x_i - \xi}{\omega}$. Using the error function, this can be expanded to:

$$y_i = \frac{2}{\omega} \frac{1}{\sqrt{2\pi}} e^{-\frac{\left(\frac{x_i - \xi}{\omega}\right)^2}{2}} \frac{1}{2} \left[1 + \frac{2}{\sqrt{\pi}} \int_0^{\gamma \frac{x_i - \xi}{\omega\sqrt{2}}} e^{-t^2} dt \right]$$

UPL Calculation for Gamma Data

In the case of gamma distributed data, the MACT floor limit is an UPL calculated as the α -level quantile of the average of m new emission runs, where the probability density, y_i , of gamma distributed emissions, x_i , is dependent on parameters λ , the shape, and τ , the rate:

$$y_i = \frac{\tau^\lambda}{\Gamma(\lambda)} x_i^{\lambda-1} e^{-\tau x_i}$$

*Note that Γ in this case is the gamma *function* not the Gamma probability distribution.

UPL Calculation for Beta Data

In the case of beta distributed data, the MACT floor limit is an UPL calculated as the α -level quantile of the average of m new emission runs, where the probability density, y_i , of beta distributed emissions, x_i , is dependent on parameters λ and τ :

$$Y_i = \frac{\Gamma(\lambda + \tau)}{\Gamma(\lambda)\Gamma(\tau)} x_i^{\lambda-1} (1 - x_i)^{\tau-1}$$

*Note that Γ in this case is the gamma *function* not the Gamma probability distribution.

Adjustment for Below Detection Level Emissions Data

To account for the effect of measurement imprecision associated with the data in the ICR databases (that include method detection level data), a protocol specified by the EPA was used. The procedure for determining a representative detection level (RDL) begins with identifying all of the available reported pollutant-specific method detection levels for the best performing units regardless of any subcategory (e.g., existing or new, fuel type, etc.). From that combined pool of data, we calculate the arithmetic mean value. By limiting the data set to those tests used to establish the floor or emissions limit (i.e., best performers), the EPA believes that the result is representative also of the best performing testing companies and laboratories. The EPA also believes that the outcome should minimize the effect of a test(s) with an inordinately high method detection level (e.g., the sample volume was too small, the laboratory technique was insufficiently sensitive, or the procedure for determining the detection level was other than that specified). The EPA then calls the resulting mean of the method detection levels the RDL, and the Agency considers it characteristic of accepted source emissions measurement performance.

The second step in the process is to calculate three times the RDL to compare with the calculated floor or emissions limit. We use the multiplication factor of 3. For comparing to the floor, if three times the RDL were less than the calculated floor or emissions limit (e.g., calculated from the UPL), we would conclude that measurement variability was adequately addressed. The calculated floor or emissions limit would need no adjustment. If, on the other hand, the value equal to three times the RDL were greater than the UPL, we would conclude that the calculated floor or emissions limit does not account entirely for measurement variability. If indicated, we substituted the value equal to three times the RDL for the calculated MACT floor UPL emissions level.

Add any RDL content specific to your source category

Table 1: Here is an example of a table in case you need one

	Existing MACT floor (old units)	Existing MACT floor (new units)	New MACT floor
Subcategory 1	HAP 1, HAP 2	HAP 1, HAP 2	HAP 1, HAP 2, X, Y, Z
Subcategory 2	HAP 1	HAP 1	HAP 1, X, Y, Z
Subcategory 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3, X, Y, Z
Subcategory 4	None	None	None
Subcategory 5	None	None	X, Y, Z

MACT Floor Data Set

describe any data that was excluded here and why

Tables with the full data sets used for each MACT floor analysis can be found listed in Appendix A.

Existing Source Chemical X for Size subcategory A

The final data set for the Chemical X Existing source MACT floor for Size subcategory A included 63 test runs from 5 sources. The best performer for the new source calculation was F with an average emission of 0.3694 (ppm) from 63 test runs.

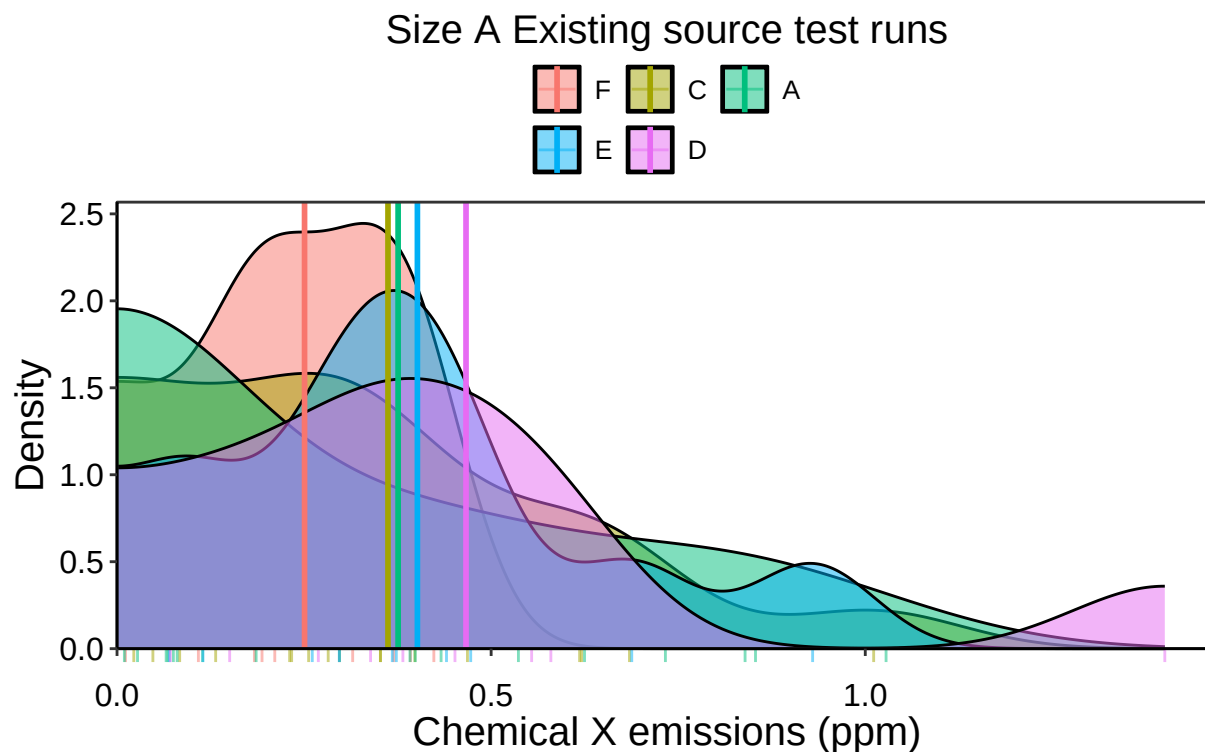


Figure 1: Observation density of emissions by source type. Source averages are indicated by vertical lines.

Existing Source Chemical X for Size subcategory B

The final data set for the Chemical X Existing source MACT floor for Size subcategory B included 64 test runs from 5 sources. The best performer for the new source calculation was F with an average emission of 0.3891 (ppm) from 64 test runs.

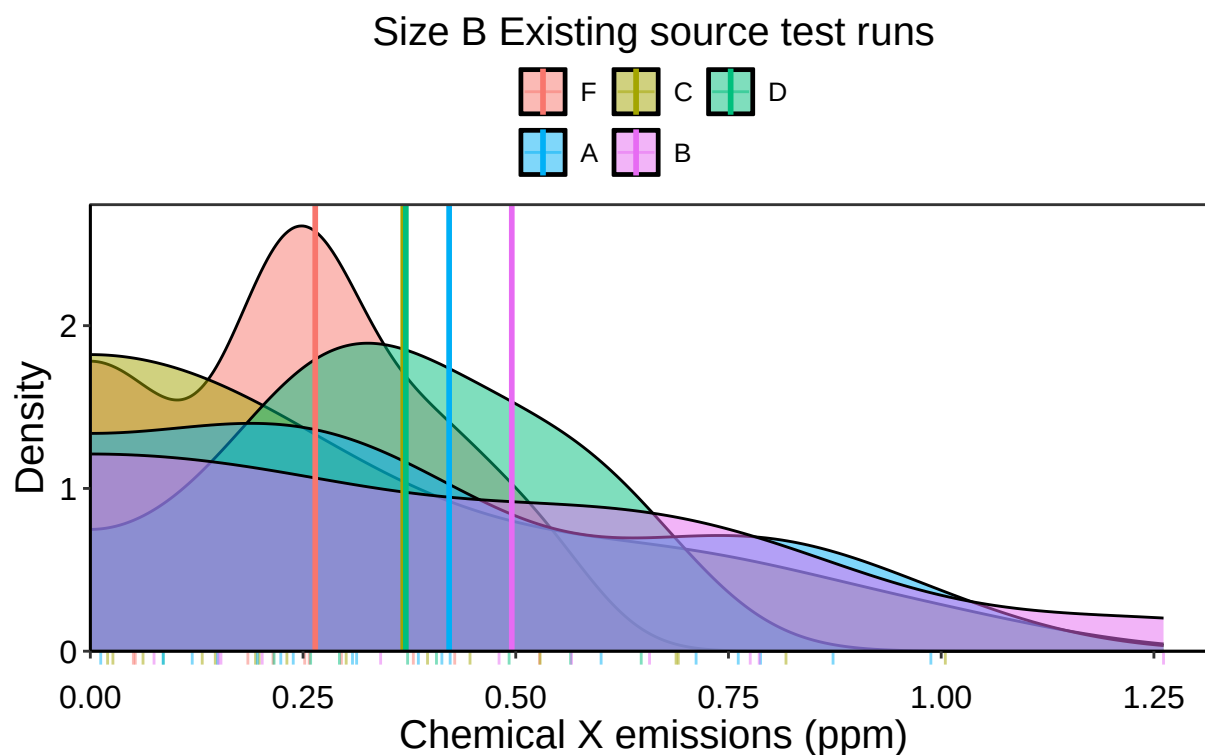


Figure 2: Observation density of emissions by source type. Source averages are indicated by vertical lines.

New Source Chemical X for Size subcategory B

The final data set for the Chemical X New source MACT floor for Size subcategory B included 10 test runs from 1 sources. The best performer for the new source calculation was F with an average emission of 0.2643 (ppm) from 10 test runs.

Existing Source Chemical Y

The final data set for the Chemical Y Existing source MACT floor included 128 test runs from 5 sources. The best performer for the new source calculation was E with an average emission of 3.984 (mg/m³/s) from 128 test runs.

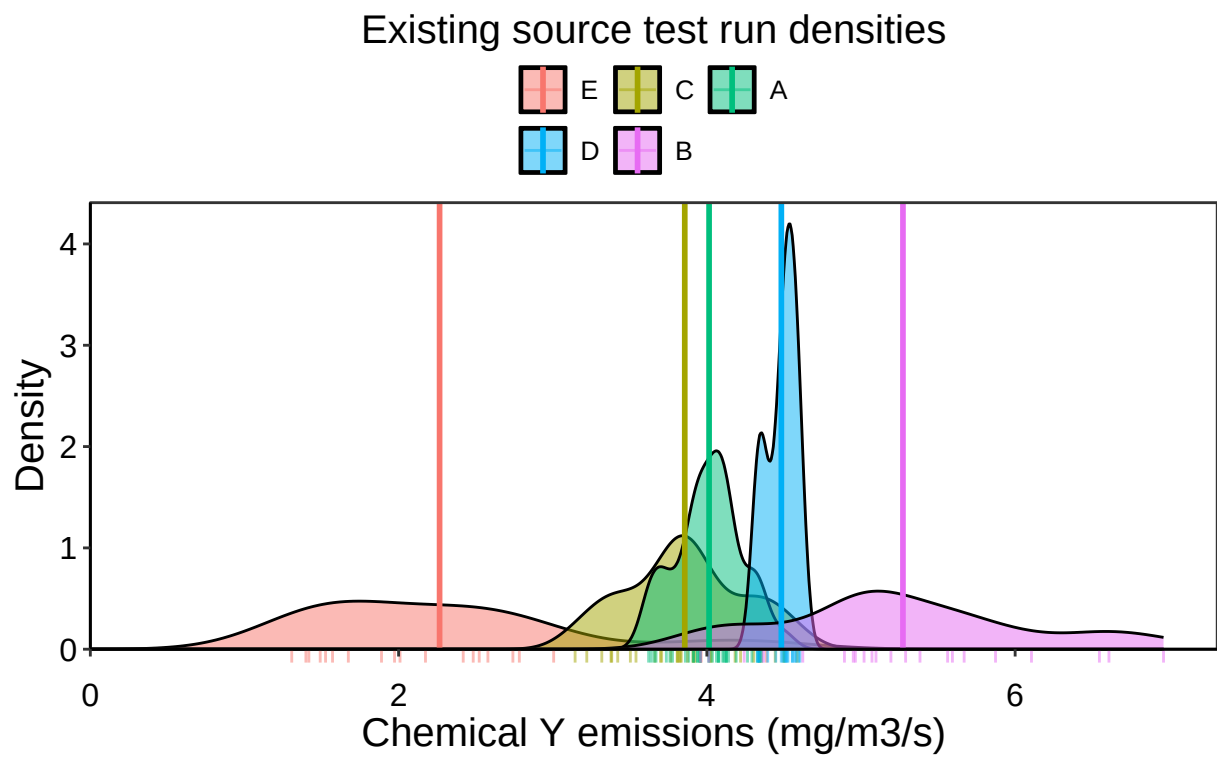


Figure 3: Observation density of emissions by source type. Source averages are indicated by vertical lines.

MACT Floor Analysis Results

Existing Source Chemical X for Size subcategory A

The distribution that best represents the Existing source data set for Chemical X for Size subcategory A was the Skewed distribution, with a corresponding UPL of 0.7022 (ppm). The Skewed distribution was the best fit distribution with a SSE of 0.6925, 63 out of 63 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 0.99.

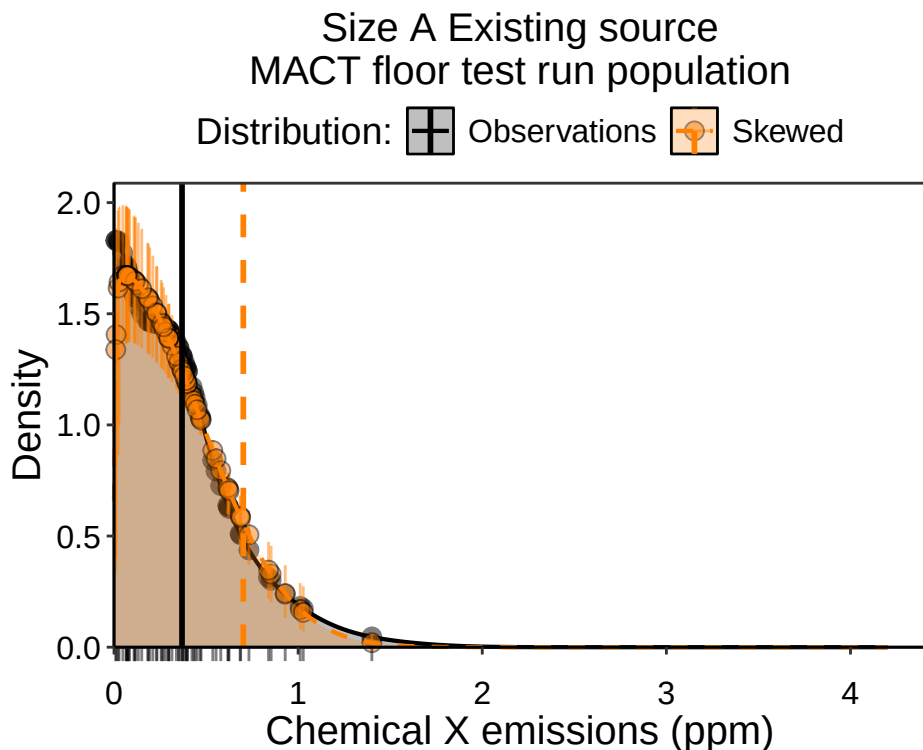


Figure 4: Observation density of emissions for population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.0546. For further details see the R code in Appendix B.

The Skewed distribution's parameters converged well, passing all Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below) and an approximately normal posterior distributions around the converged value (right figures below).

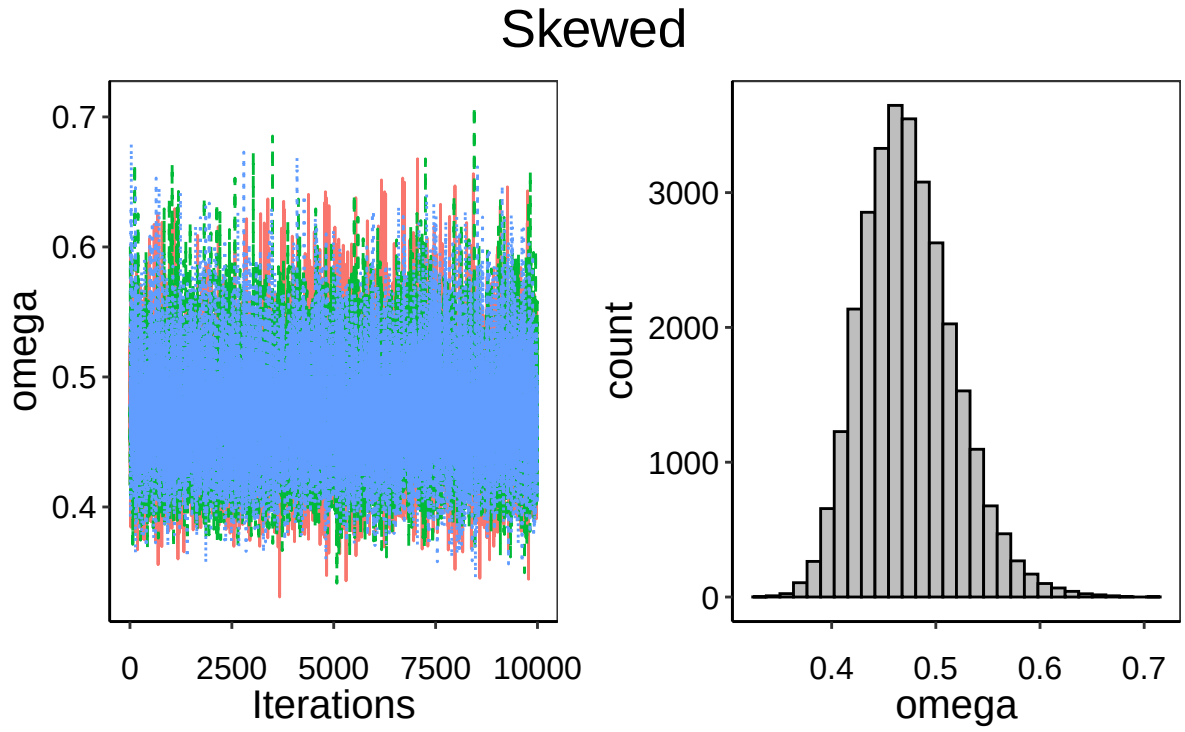


Figure 5: Convergence figures for distribution parameters.

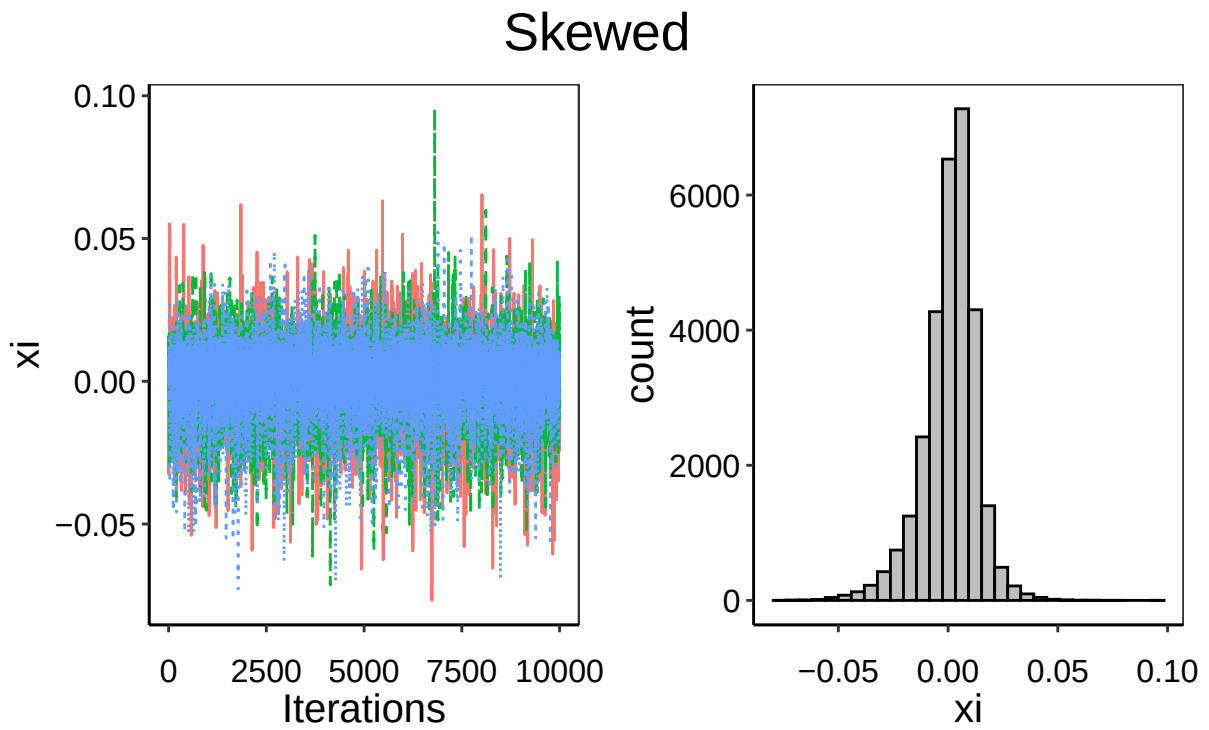


Figure 6: Convergence figures for distribution parameters.

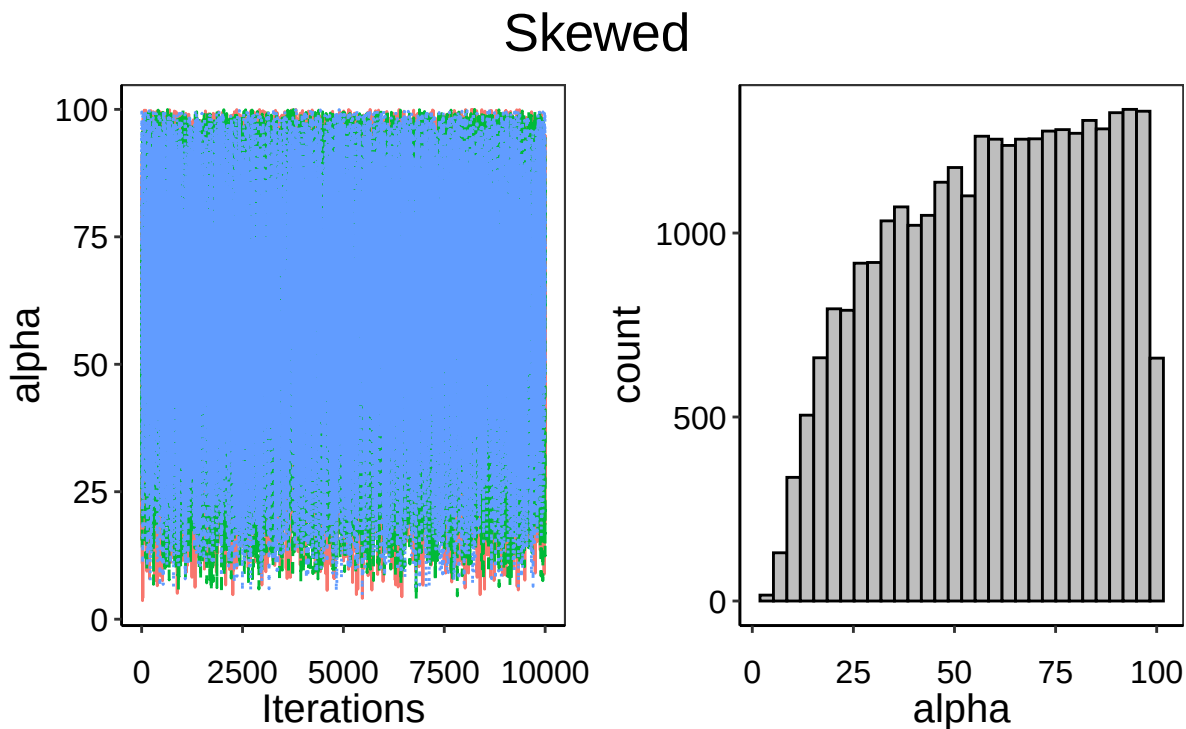


Figure 7: Convergence figures for distribution parameters.

Existing Source Chemical X for Size subcategory B

The distribution that best represents the Existing source data set for Chemical X for Size subcategory B was the Skewed distribution, with a corresponding UPL of 0.7583 (ppm). The Skewed distribution was the best fit distribution with a *SSE* of 0.3373, 59 out of 64 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 0.99.

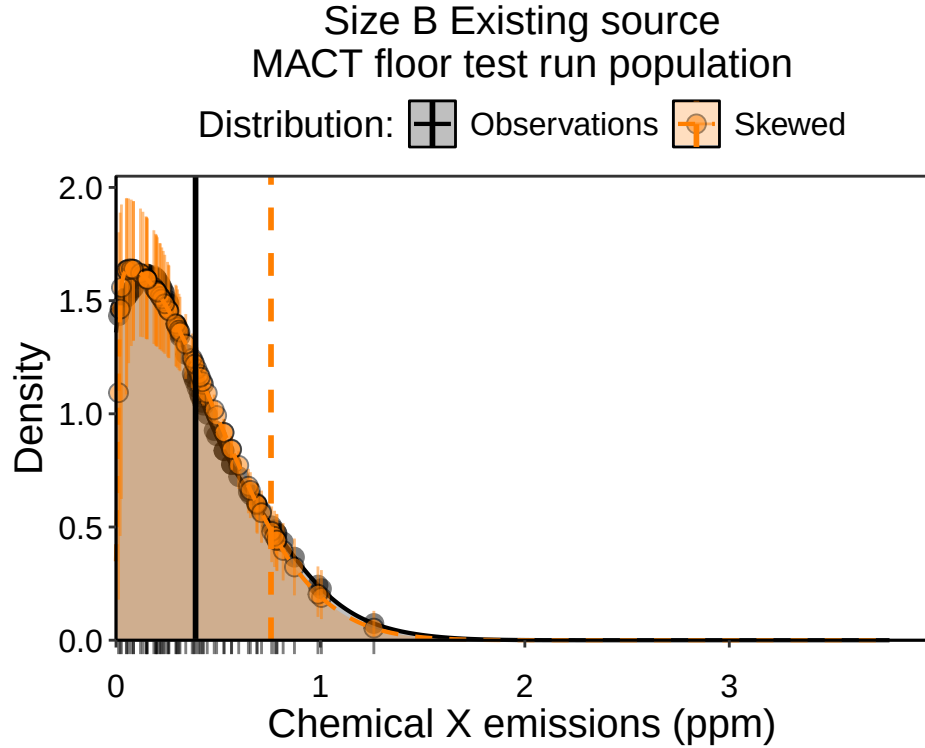


Figure 8: Observation density of emissions for population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.053. For further details see the R code in Appendix B.

The Skewed distribution's parameters converged well, passing all Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below) and an approximately normal posterior distributions around the converged value (right figures below).

Skewed

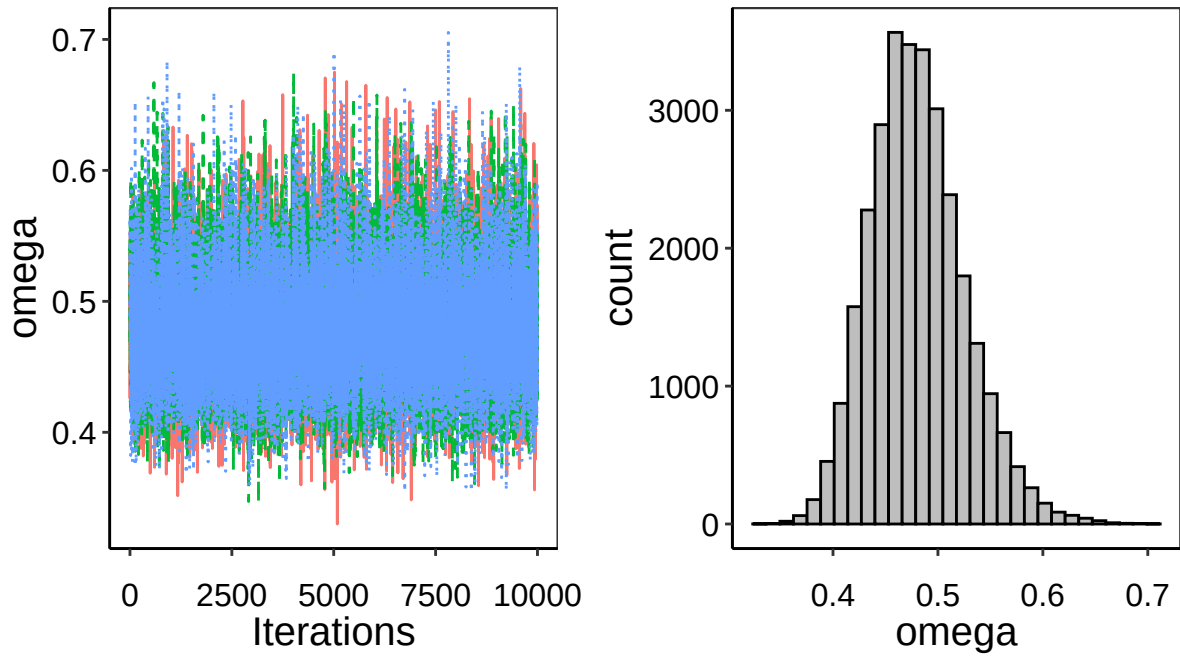


Figure 9: Convergence figures for distribution parameters.

Skewed

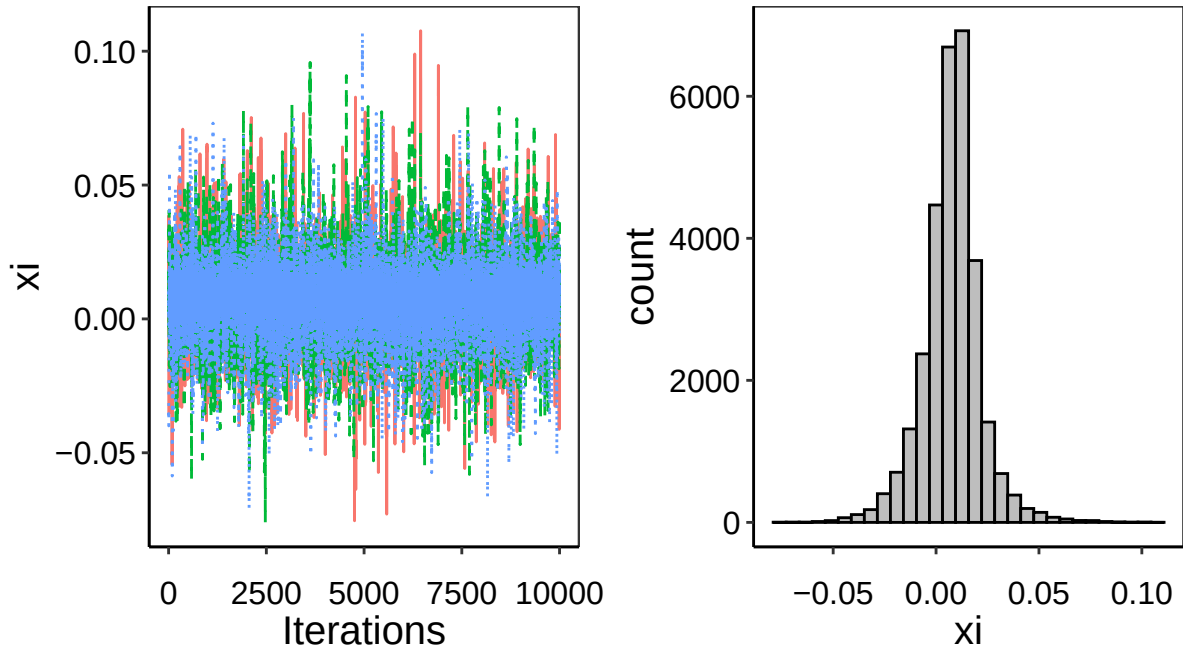


Figure 10: Convergence figures for distribution parameters.

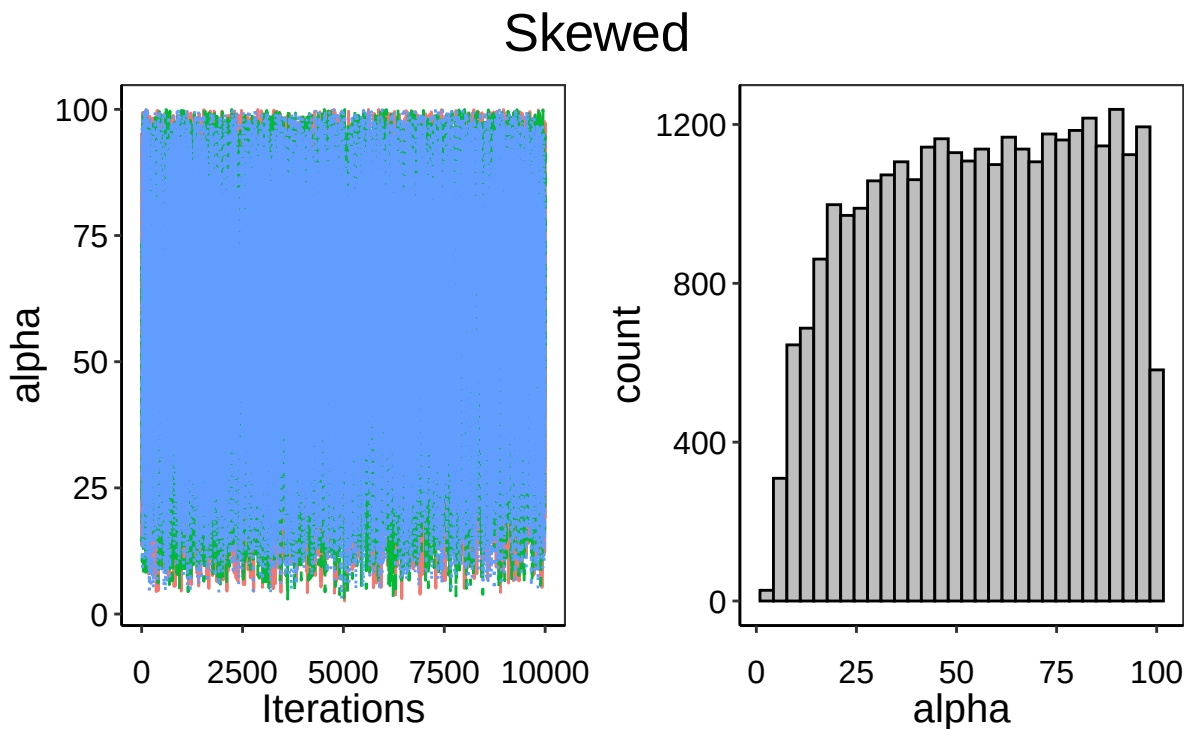


Figure 11: Convergence figures for distribution parameters.

New Source Chemical X for Size subcategory B

The distribution that best represents the New source data set for Chemical X for Size subcategory B was the Skewed distribution, with a corresponding UPL of 0.6952 (ppm). The Skewed distribution was the best fit distribution with a *SSE* of 0.3145, 10 out of 10 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 0.89.

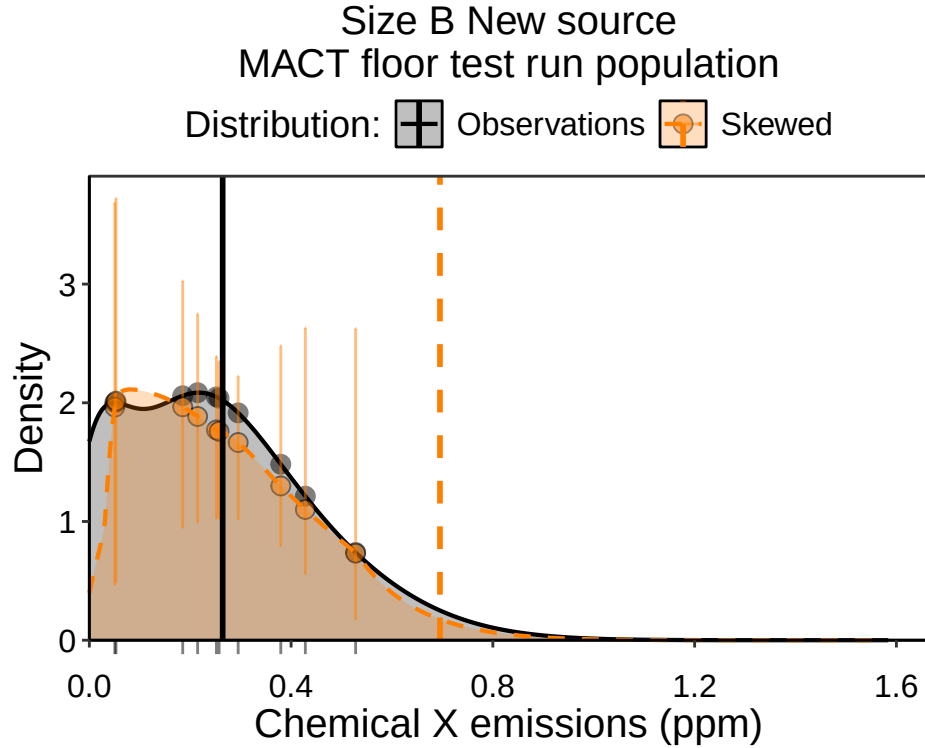


Figure 12: Observation density of emissions for population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.0608. For further details see the R code in Appendix B.

The Skewed distribution's parameters did not converge, failing some Gelman-Rubin diagnostic checks. Check the figures below to see if MCMC chains (left figures below) are well-mixed and posterior distributions are approximately normal around the converged value (right figures below). Results from this distribution model should not be used until the convergence issues is corrected. Consider a more appropriate choice of distribution, or manually specifying better priors and likelihood limits.

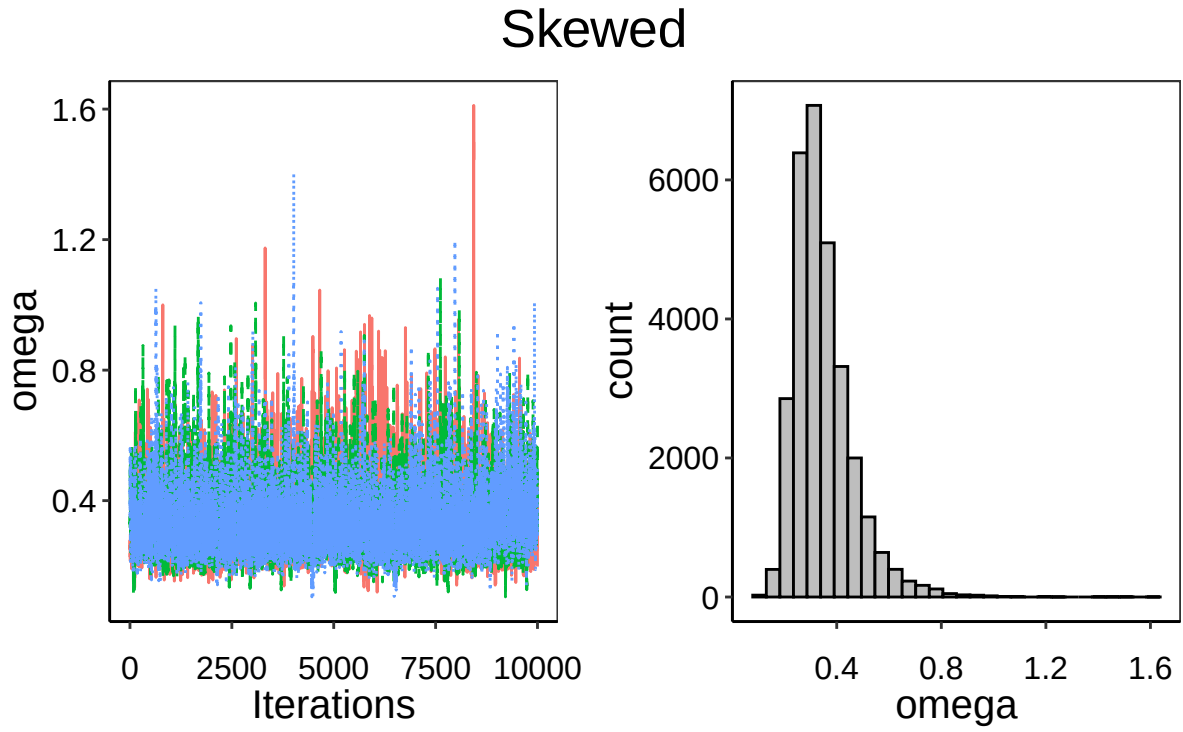


Figure 13: Convergence figures for distribution parameters.

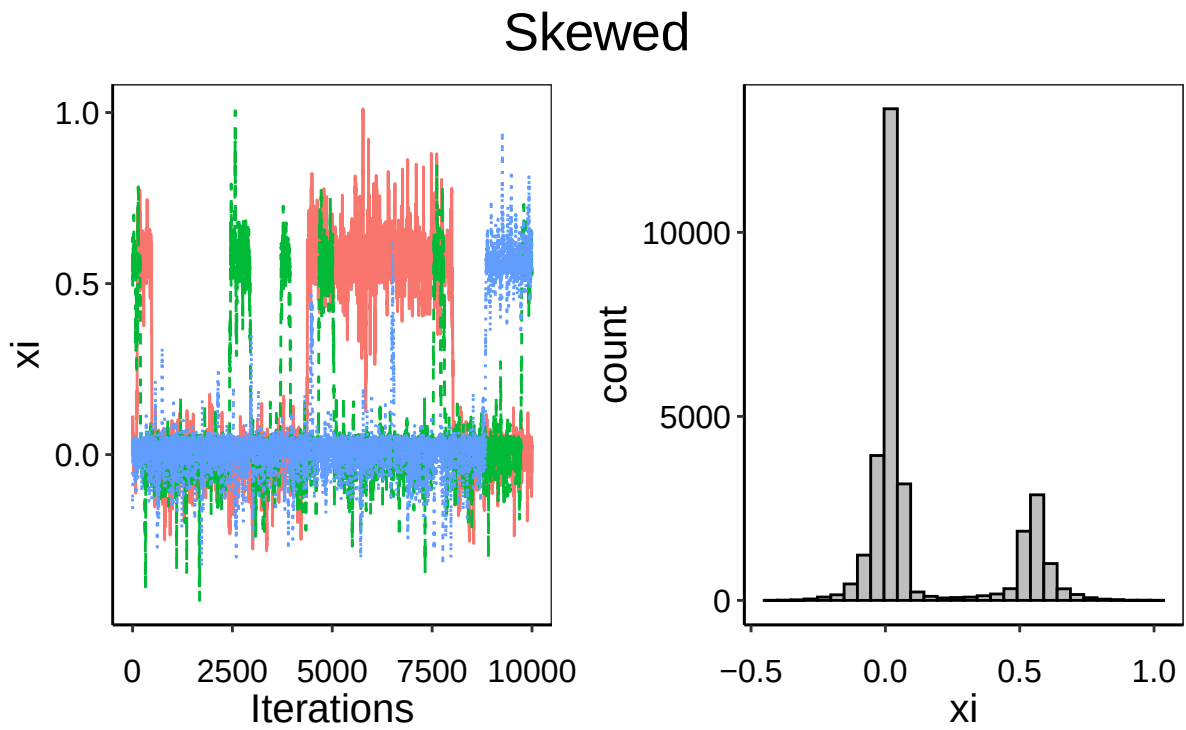


Figure 14: Convergence figures for distribution parameters.

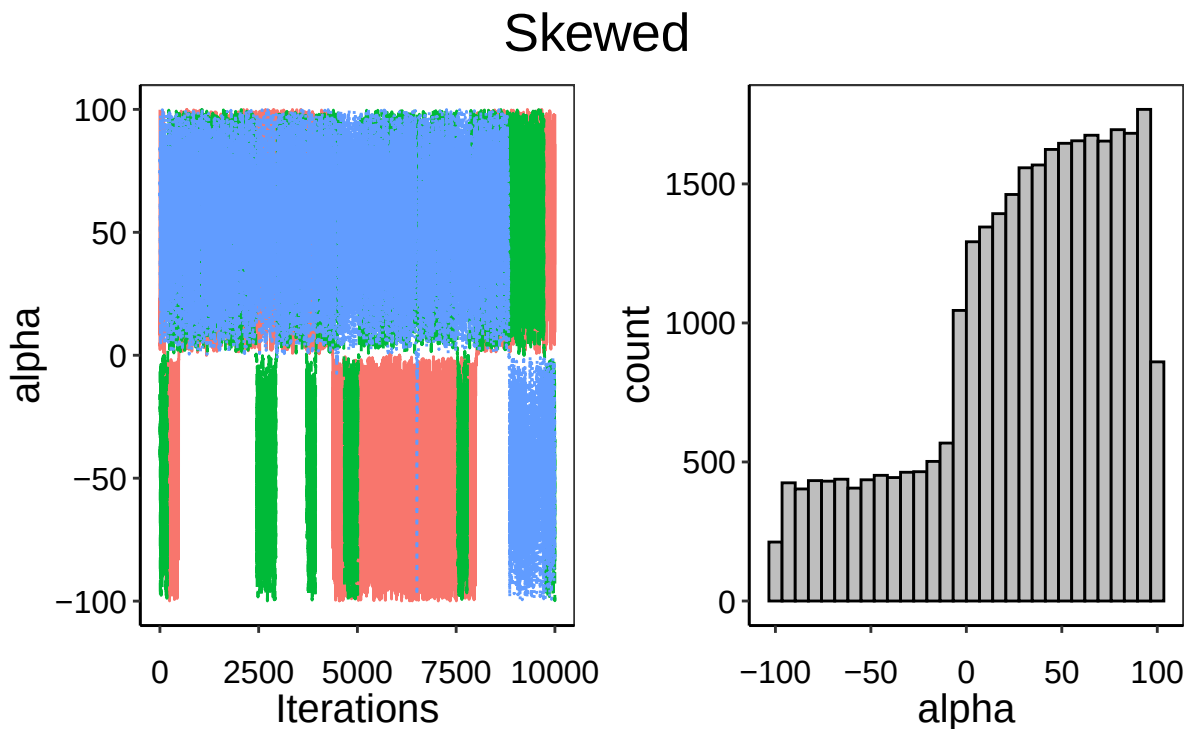


Figure 15: Convergence figures for distribution parameters.

Existing Source Chemical Y

The distribution that best represents the Existing source data set for Chemical Y was the Normal distribution, with a corresponding UPL of 5.406 (mg/m³/s). The Normal distribution was the best fit distribution with a *SSE* of 0.07574, 114 out of 128 emission values within the 95% CI of predictions, and the probability density function integrated to approximately 1.

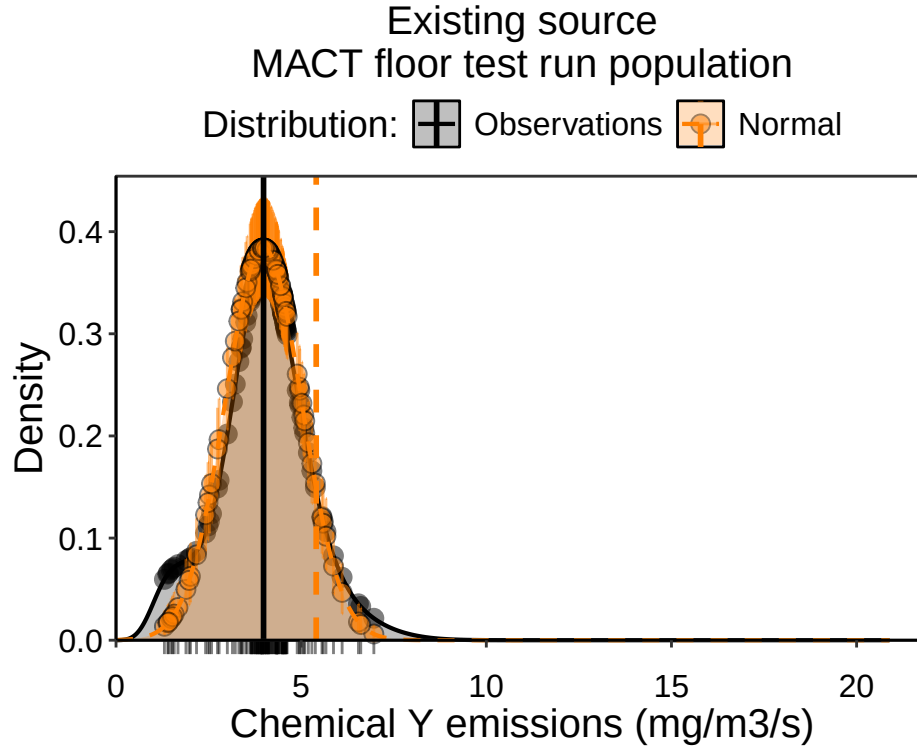


Figure 16: Observation density of emissions for population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.147. For further details see the R code in Appendix B.

The Normal distribution's parameters converged well, passing all Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below) and an approximately normal posterior distributions around the converged value (right figures below).

Normal

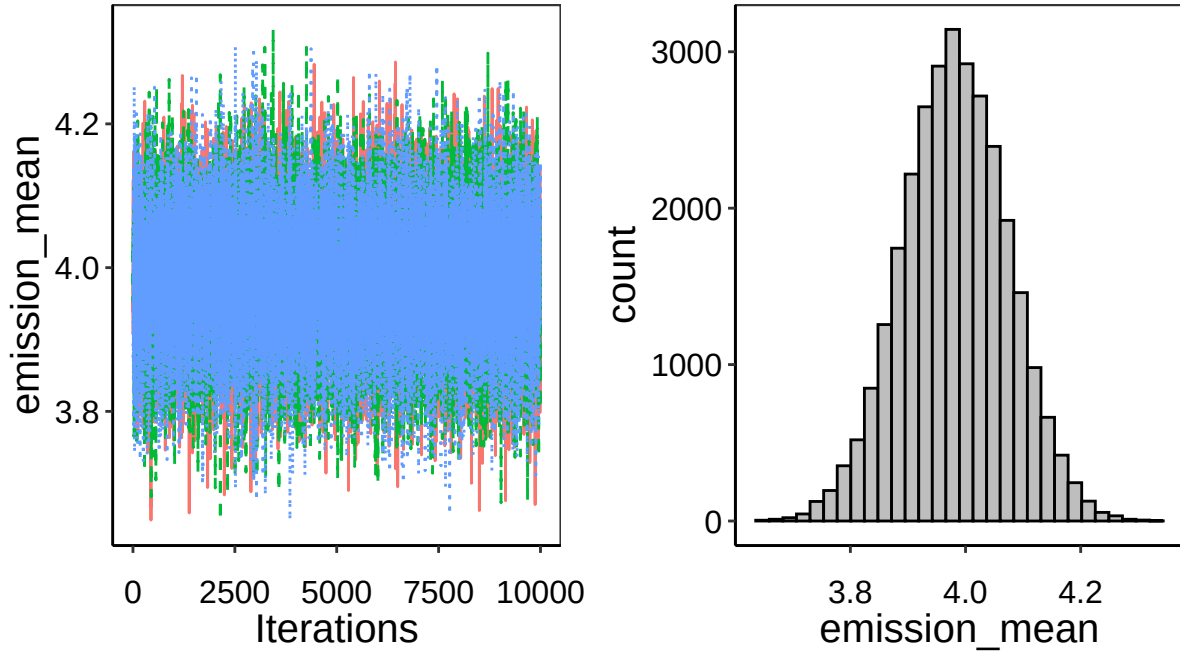


Figure 17: Convergence figures for distribution parameters.

Normal

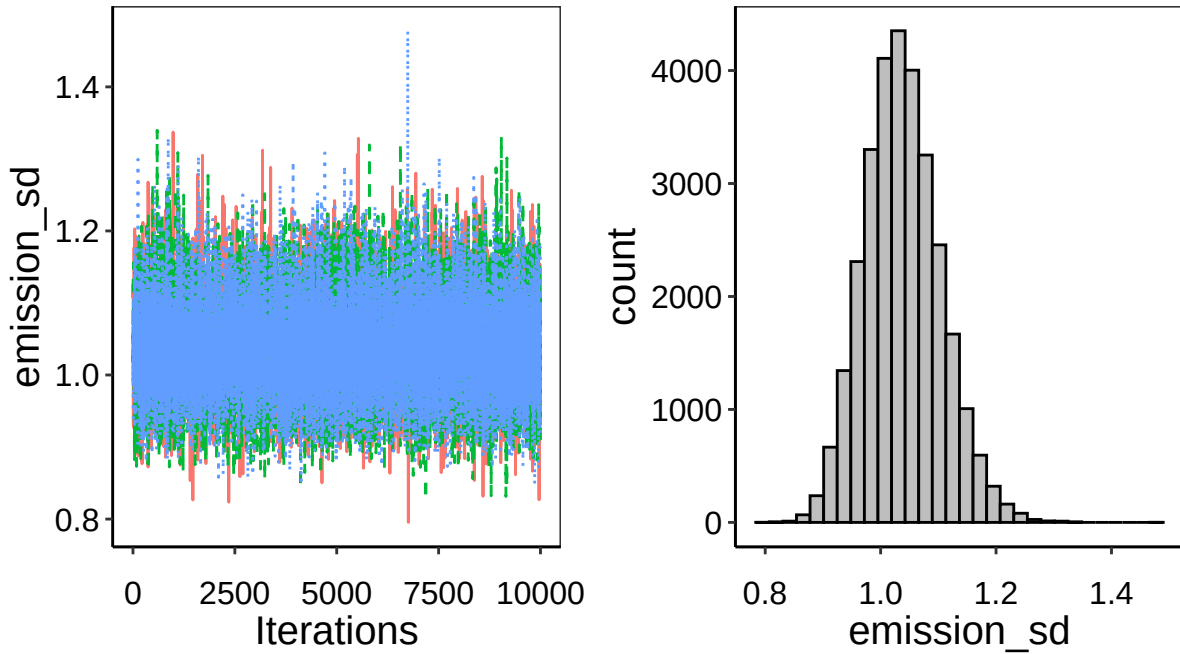


Figure 18: Convergence figures for distribution parameters.

UPL Calculation Summary

The table below includes all UPL calculations for Example Industry MACT floor analyses.

Table 2: Upper Predictive Limit (UPL) results for MACT floors in (measurement units)

HAP	SubcategoryName	SubcategoryLevel	Source Type	UPL	Significance	No. FutureRuns
Chemical X	Size	A	Existing	0.7022	0.99	3
Chemical X	Size	B	Existing	0.7583	0.99	3
Chemical X	Size	B	New	0.6952	0.99	3
Chemical Y	NA	NA	Existing	5.4060	0.99	3

Settings used in Bayesian MCMC

Existing Source Chemical X for Size subcategory A

The selected distribution for the Existing source data set for Chemical X for Size subcategory A was the Skewed distribution, with the likelihood truncated such that it was zero outside the bounds of (0, 4.2) emissions. The parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence are listed in the table below, followed by the random number generator (RNG) state that was used.

Table 3: Parameter information for the Skewed distribution UPL calculation for Existing source Chemical X for Size subcategory A

Parameter	Prior	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Omega	~dunif (0, 4.2)	-0.00902	-0.0135	-0.00451	1	Yes
Xi	~dnorm (0, 0.00238)	0.47500	0.2370	0.71200	1	Yes
Alpha	~dunif (-100, 100)	27.90000	13.9000	41.80000	1	Yes

Table 4: Random number generator (RNG) names and states used in MCMC chains

MCMC Chain	Name	State
Chain 1	base::Wichmann-Hill	seed = 5
Chain 2	base::Marsaglia-Multicarry	seed = 12
Chain 3	base::Super-Duper	seed = 151

Existing Source Chemical X for Size subcategory B

The selected distribution for the Existing source data set for Chemical X for Size subcategory B was the Skewed distribution, with the likelihood truncated such that it was zero outside the bounds of (0, 3.78)

emissions. The parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence are listed in the table below, followed by the random number generator (RNG) state that was used.

Table 5: Parameter information for the Skewed distribution UPL calculation for Existing sourceChemical X for Size subcategory B

Parameter	Prior	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Omega	$\sim$ dunif (0, 3.78)	0.0389	0.0583	0.0194	1	Yes
Xi	$\sim$ dnorm (0, 0.00264)	0.4480	0.2240	0.6720	1	Yes
Alpha	$\sim$ dunif (-100, 100)	4.8500	2.4200	7.2700	1	Yes

Table 6: Random number generator (RNG) names and states used in MCMC chains

MCMC Chain	Name	State
Chain 1	base::Wichmann-Hill	seed = 5
Chain 2	base::Marsaglia-Multicarry	seed = 12
Chain 3	base::Super-Duper	seed = 151

New Source Chemical X for Size subcategory B

The selected distribution for the New source data set for Chemical X for Size subcategory B was the Skewed distribution, with the likelihood truncated such that it was zero outside the bounds of (0, 1.58) emissions. The parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence are listed in the table below, followed by the random number generator (RNG) state that was used.

Table 7: Parameter information for the Skewed distribution UPL calculation for New sourceChemical X for Size subcategory B

Parameter	Prior	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Omega	$\sim$ dunif (0, 1.58)	0.163	0.2450	0.0816	1.00	Yes
Xi	$\sim$ dnorm (0, 0.00631)	0.183	0.0916	0.2750	1.12	Weak convergence
Alpha	$\sim$ dunif (-100, 100)	0.957	0.4790	1.4400	1.08	Yes

Table 8: Random number generator (RNG) names and states used in MCMC chains

MCMC Chain	Name	State
Chain 1	base::Wichmann-Hill	seed = 5
Chain 2	base::Marsaglia-Multicarry	seed = 12
Chain 3	base::Super-Duper	seed = 151

MCMC Chain	Name	State
------------	------	-------

Existing Source Chemical Y

The selected distribution for the Existing source data set for Chemical Y was the Normal distribution, with the likelihood truncated such that it was zero outside the bounds of (0, 20.9) emissions. The parameter prior distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for convergence are listed in the table below, followed by the random number generator (RNG) state that was used.

Table 9: Parameter information for the Normal distribution UPL calculation for Existing sourceChemical Y

Parameter	Prior	Chain 1 initial value	Chain 2 initial value	Chain 3 initial value	Gelman-Rubin diagnostic	Converged?
Mean	$\sim \text{dunif}(0, 20.9)$	3.98	5.980	1.99	1	Yes
Standard deviation	$\sim \text{dunif}(0.00103, 1030)$	1.03	0.513	1.54	1	Yes

Table 10: Random number generator (RNG) names and states used in MCMC chains

MCMC Chain	Name	State
Chain 1	base::Wichmann-Hill	seed = 5
Chain 2	base::Marsaglia-Multicarry	seed = 12
Chain 3	base::Super-Duper	seed = 151

References

- Dragan Doric, et al. 2007. On measuring skewness and kurtosis. Springer Science + Buisness Media B. V. 2007.
- Hobbs, N. Thompson and Mevin B. Hooten. 2015. “Bayesian Models: A statistical Primer for Ecologists”. Princeton University Press.
- Renault O. & O. Scaillet. On the way to recovery: A parametric bias free estimation of recovery rate densities. 2004. Journal of Banking and Finance. 28:12 p. 2915-2931.

Appendix A: Example Industry MACT Floor Data Tables

Existing source MACT floor analysis data set for HAP Chemical X for Size subcategory A:

Table 11: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
0.423457524	F	-0.85930207
0.297248523	F	-1.21318671
0.183629417	F	-1.69483559
0.193793366	F	-1.64096281
0.011319561	F	-4.48122299
0.314972832	F	-1.15526889
0.369923676	F	-0.99445858
0.108745685	F	-2.21874329
0.392228734	F	-0.93591010
0.210861879	F	-1.55655196
0.398017112	C	-0.92126028
0.684898563	C	-0.37848453
0.351969284	C	-1.04421137
0.468336739	C	-0.75856771
0.256083951	C	-1.36224995
0.047938254	C	-3.03784147
1.011243319	C	0.01118058
0.131841595	C	-2.02615411
0.233375757	C	-1.45510543
0.620988130	C	-0.47644331
0.230697080	C	-1.46664977
0.083732778	C	-2.48012477
0.618233933	C	-0.48088836
0.352198298	C	-1.04356092
0.282250804	C	-1.26495923
0.022425698	C	-3.79754775
0.065657791	A	-2.72329901
0.297382273	A	-1.21273685
0.027433451	A	-3.59599217
0.075277667	A	-2.58657178
0.398608739	A	-0.91977495
0.536446013	A	-0.62278935
0.732912157	A	-0.31072942
0.114383344	A	-2.16819981
0.009847679	A	-4.62051949
0.853306446	A	-0.15863654
0.391558860	A	-0.93761943
0.068143426	A	-2.68614059
0.624795018	A	-0.47033165
0.080409694	A	-2.52062054
0.433115640	A	-0.83675052
0.185903732	A	-1.68252631
0.839401045	A	-0.17506668
1.027793905	A	0.02741467
0.367923876	E	-0.99987922
0.114854649	E	-2.16408787

chemX	boiler units	log emissions
0.373251720	E	-0.98550223
0.929656507	E	-0.07294011
0.440352699	E	-0.82017928
0.070091324	E	-2.65795626
0.297020332	E	-1.21395468
0.260939213	E	-1.34346780
0.687909020	E	-0.37409869
0.472692409	E	-0.74931040
0.451683720	D	-0.79477308
0.150544410	D	-1.89349716
0.269062607	D	-1.31281119
0.579810158	D	-0.54505454
0.554135158	D	-0.59034665
0.382055680	D	-0.96218892
1.400347741	D	0.33672059
0.071206206	D	-2.64217530
0.338863859	D	-1.08215685

Existing source MACT floor analysis data set for HAP Chemical X for Size subcategory B:

Table 12: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
0.18516235	F	-1.686522266
0.25194760	F	-1.378534134
0.21470447	F	-1.538492764
0.29507380	F	-1.220529795
0.37961787	F	-0.968590124
0.25725330	F	-1.357694057
0.05054736	F	-2.984844580
0.42826767	F	-0.848006875
0.52795715	F	-0.638740150
0.05280095	F	-2.941226172
0.52865548	C	-0.637418318
0.02032185	C	-3.896058371
0.68839100	C	-0.373398283
0.44622405	C	-0.806934103
0.06193868	C	-2.781610383
0.23103245	C	-1.465197124
0.81760572	C	-0.201375066
0.19417074	C	-1.639017424
0.02655390	C	-3.628578536
0.19842015	C	-1.617368502
0.30068897	C	-1.201678871
1.00485388	C	0.004842142
0.69102236	C	-0.369583101
0.39631653	C	-0.925542064
0.13149748	C	-2.028767561
0.14686333	C	-1.918252853
0.56445382	D	-0.571896706

chemX	boiler units	log emissions
0.21594011	D	-1.532754155
0.49219149	D	-0.708887423
0.64745805	D	-0.434701281
0.40681661	D	-0.899392772
0.37290786	D	-0.986423911
0.25891152	D	-1.351268901
0.29277314	D	-1.228357246
0.08579818	D	-2.455757543
0.78765570	A	-0.238694210
0.38554450	A	-0.953098653
0.01222692	A	-4.404115201
0.22384561	A	-1.496798706
0.60029987	A	-0.510325964
0.42287278	A	-0.860683895
0.41317618	A	-0.883881184
0.30810868	A	-1.177302711
0.11974922	A	-2.122355523
0.98788204	A	-0.012191979
0.14947349	A	-1.900636220
0.31296418	A	-1.161666536
0.19628648	A	-1.628180039
0.87289515	A	-0.135939831
0.08559653	A	-2.458110592
0.23847315	A	-1.433498537
0.76148833	A	-0.272480429
0.71194168	A	-0.339759275
0.77556127	B	-0.254168298
0.56558983	B	-0.569886142
1.26132825	B	0.232165328
0.34118597	B	-1.075327572
0.65723420	B	-0.419714863
0.07489268	B	-2.591699190
0.78611526	B	-0.240651857
0.20198364	B	-1.599568590
0.15076012	B	-1.892065341
0.48037758	B	-0.733182861
0.15351979	B	-1.873925782

New source MACT floor analysis data set for HAP Chemical X for Size subcategory B:

Table 13: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
0.18516235	F	-1.6865223
0.25194760	F	-1.3785341
0.21470447	F	-1.5384928
0.29507380	F	-1.2205298
0.37961787	F	-0.9685901
0.25725330	F	-1.3576941
0.05054736	F	-2.9848446

chemX	boiler units	log emissions
0.42826767	F	-0.8480069
0.52795715	F	-0.6387402
0.05280095	F	-2.9412262

Existing source MACT floor analysis data set for HAP Chemical Y :

Table 14: Top performing source emissions data used in MACT floor UPL calculations.

chemY	boiler units	log emissions
2.174424	E	0.7767640
1.673932	E	0.5151754
1.889162	E	0.6361334
2.783342	E	1.0236523
2.419206	E	0.8834394
3.006086	E	1.1006388
1.306758	E	0.2675496
1.491378	E	0.3997004
1.571521	E	0.4520438
2.009255	E	0.6977641
1.398739	E	0.3355712
1.526092	E	0.4227104
1.417813	E	0.3491157
2.523017	E	0.9254552
2.484151	E	0.9099311
2.741503	E	1.0085063
3.963096	E	1.3770257
1.973820	E	0.6799708
2.579510	E	0.9475996
4.386490	E	1.4785294
3.540232	C	1.2641922
3.382720	C	1.2186803
3.421676	C	1.2301305
4.344752	C	1.4689686
4.586216	C	1.5230554
4.303682	C	1.4594710
3.663421	C	1.2983974
3.145160	C	1.1458648
3.911287	C	1.3638665
4.356717	C	1.4717187
4.218697	C	1.4395264
3.957258	C	1.3755515
3.911109	C	1.3638210
3.378192	C	1.2173407
4.021903	C	1.3917551
3.827899	C	1.3423160
3.833986	C	1.3439050
3.699019	C	1.3080677
3.220992	C	1.1696895
3.503628	C	1.2537991
4.183033	C	1.4310365

chemY	boiler units	log emissions
3.819464	C	1.3401101
3.807323	C	1.3369264
3.706842	C	1.3101802
4.500295	C	1.5041430
3.809243	C	1.3374304
3.924598	C	1.3672639
3.874610	C	1.3544451
3.318383	C	1.1994777
3.766767	C	1.3262170
4.443461	C	1.4914336
4.029063	C	1.3935338
4.292470	A	1.4568624
4.140423	A	1.4207980
4.501422	A	1.5043934
3.621995	A	1.2870249
3.882037	A	1.3563601
3.657100	A	1.2966704
3.915800	A	1.3650198
4.062028	A	1.4016824
4.340514	A	1.4679928
3.911323	A	1.3638757
3.760281	A	1.3244936
3.938524	A	1.3708060
4.124211	A	1.4168747
4.036380	A	1.3953484
4.263680	A	1.4501327
3.940218	A	1.3712361
3.670356	A	1.3002886
4.190300	A	1.4327723
3.777375	A	1.3290294
4.123393	A	1.4166764
4.102699	A	1.4116450
4.073892	A	1.4045988
4.344779	A	1.4689749
3.958771	A	1.3759336
3.737161	A	1.3183262
4.012695	A	1.3894630
3.953605	A	1.3746277
4.127012	A	1.4175537
4.326929	A	1.4648580
3.638334	A	1.2915260
3.957223	A	1.3755425
4.014073	A	1.3898065
4.109942	A	1.4134088
3.860635	A	1.3508318
4.078113	A	1.4056344
4.076282	A	1.4051854
4.354797	D	1.4712780
4.507734	D	1.5057946
4.555990	D	1.5164427
4.492505	D	1.5024105
4.578270	D	1.5213211

chemY	boiler units	log emissions
4.482733	D	1.5002329
4.394871	D	1.4804381
4.572945	D	1.5201574
4.526267	D	1.5098974
4.554366	D	1.5160863
4.604106	D	1.5269485
4.519751	D	1.5084568
4.337042	D	1.4671926
4.512104	D	1.5067636
4.336258	D	1.4670117
4.446896	D	1.4922063
4.595368	D	1.5250488
4.327935	D	1.4650905
5.289483	B	1.6657205
5.194230	B	1.6475484
6.546733	B	1.8789661
5.872839	B	1.7703382
6.962961	B	1.9406049
5.071315	B	1.6236002
4.242131	B	1.4450658
4.963653	B	1.6021419
5.382787	B	1.6832063
5.097371	B	1.6287249
4.949946	B	1.5993767
3.959365	B	1.3760836
4.622315	B	1.5308957
4.368199	B	1.4743507
5.562062	B	1.7159690
5.593076	B	1.7215294
6.608701	B	1.8883871
5.019913	B	1.6134126
6.106131	B	1.8092933
4.892633	B	1.5877306
4.012346	B	1.3893762
5.669356	B	1.7350756

Appendix B: R Function Code

Disclaimer

The United States Environmental Protection Agency (EPA) GitHub project code is provided on an “as is” basis and the user assumes responsibility for its use. EPA has relinquished control of the information and no longer has responsibility to protect the integrity, confidentiality, or availability of the information. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by EPA. The EPA seal and logo shall not be used in any manner to imply endorsement of any commercial product or activity by EPA or the United States Government.

MACT_existing()

```
#> function (data, emissions, sources, CAA_section = 112, national_N = NA)
#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]],
#>     sources = data[[sources]])
#>   if (!is.numeric(data_temp$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   if (any(data_temp$emissions < 0)) {
#>     warning("emissions data contain negative values")
#>   }
#>   if (any(data_temp$emissions == 0)) {
#>     warning("emissions data contain zero values and have been removed")
#>     data_temp = subset(data_temp, data_temp$emissions !=
#>       0)
#>   }
#>   dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>     .by = "sources")
#>   n_sources = length(unique(data_temp$sources))
#>   if (CAA_section == 129) {
#>     if (is.na(national_N)) {
#>       stop("Must provide total number of sources as national_N")
#>     }
#>     n_topsources = ceiling(0.12 * national_N)
#>     if (n_sources < n_topsources) {
#>       n_topsources = n_sources
#>     }
#>     top_list = dat_means[order(dat_means$means, decreasing = F),
#>       ]
#>     top_list = top_list$sources[1:n_topsources]
#>     dat_top = subset(data_temp, data_temp$sources %in% top_list)
#>   }
#>   else if (CAA_section == 112) {
#>     if (n_sources >= 30) {
#>       n_topsources = ceiling(0.12 * n_sources)
#>       top_list = dat_means[order(dat_means$means, decreasing = F),
#>         ]
#>       top_list = top_list$sources[1:n_topsources]
#>       dat_top = subset(data_temp, data_temp$sources %in%
```

```

#>         top_list)
#>     }
#>     else if (n_sources < 30) {
#>         n_topsources = 5
#>         top_list = dat_means[order(dat_means$means, decreasing = F),
#>                                ]
#>         top_list = top_list$sources[1:n_topsources]
#>         dat_top = subset(data_temp, data_temp$sources %in%
#>                           top_list)
#>     }
#> }
#> dat_topmeans = dplyr::summarize(dat_top, means = mean(emissions),
#>                                .by = "sources", counts = dplyr::n())
#> dat_topmeans$sources = as.factor(dat_topmeans$sources)
#> dat_topmeans$sources = forcats::fct_reorder(dat_topmeans$sources,
#>       dat_topmeans$means, .desc = FALSE)
#> dat_top$sources = factor(dat_top$sources, levels = levels(dat_topmeans$sources))
#> dat_topmeans = dplyr::arrange(dat_topmeans, means)
#> dat_top = dplyr::arrange(dat_top, sources)
#> dat_top$sources = droplevels(dat_top$sources)
#> if (!is.character(emissions)) {
#>     emissions = colnames(data)[emissions]
#> }
#> if (!is.character(sources)) {
#>     sources = colnames(data)[sources]
#> }
#> names(dat_top) = c(emissions, sources)
#> return(dat_top)
#> }
#> <bytecode: 0x00000282f8021f08>
#> <environment: namespace:UPLforOAR>

```

MACT_new()

```

#> function (data, emissions, sources)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>                                sources = data[[sources]])
#>     if (!is.numeric(data_temp$emissions)) {
#>         stop("Emissions must be numeric vector")
#>     }
#>     if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>         stop("Sources must be a character or factor vector")
#>     }
#>     if (any(data_temp$emissions < 0)) {
#>         warning("emissions data contain negative values")
#>     }
#>     if (any(data_temp$emissions == 0)) {
#>         warning("emissions data contain zero values and have been removed")
#>         data_temp = subset(data_temp, data_temp$emissions !=
#>                             0)
#>     }
#>     dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>                                .by = "sources")

```

```

#>   top_list = dat_means[order(dat_means$means, decreasing = F),
#>   ]
#>   top_source = top_list$sources[1]
#>   dat_top = subset(data_temp, data_temp$sources == top_source)
#>   dat_top$sources = as.factor(dat_top$sources)
#>   dat_top$sources = droplevels(dat_top$sources)
#>   if (!is.character(emissions)) {
#>     emissions = colnames(data)[emissions]
#>   }
#>   if (!is.character(sources)) {
#>     sources = colnames(data)[sources]
#>   }
#>   names(dat_top) = c(emissions, sources)
#>   return(dat_top)
#> }
#> <bytecode: 0x00000283083a1708>
#> <environment: namespace:UPLforOAR>

```

write_likelihood()

```

#> [1] "' Writes likelihood scripts for JAGS model calls"
#> [2] "' @description"
#> [3] "' This function writes an R script for JAGS to call based on the selected"
#> [4] "' distribution and prior. The priors are uninformative and"
#> [5] "' set based on emissions data, unless specified manually via"
#> [6] "' [setup_likelihood()]. The likelihood distributions are truncated to"
#> [7] "' `(minY, maxY)`, where `minY` and `maxY` can be specified or used with the"
#> [8] "' default `minY = 0` and `maxY = 3 * max(data$emissions)` in [run_likelihood()]."
#> [9] "' @param distribution Any of `Normal`, `Gamma`, `Skewed`, `Lognormal`, "
#> [10] "' or `Beta`."
#> [11] "' @param write_wd Default is `NULL`, in which case the JAGS scripts are written"
#> [12] "' into inst/JAGS folder in package directory. This is the location"
#> [13] "' [run_likelihood()] will look for the JAGS scripts assigned via"
#> [14] "' [setup_likelihood()]."
#> [15] "' @param manual_prior Default is `FALSE`, if priors should be specified"
#> [16] "' manually or be uninformative calculated from range of emissions data."
#> [17] "' @returns object `model_code`, which is a string for the written R script that"
#> [18] "' JAGS can call and the distribution used in likelihood model."
#> [19] "' @export"
#> [20] "write_likelihood = function(distribution,"
#> [21] "                                manual_prior = FALSE, write_wd = NULL){
#> [22] "   current_wd = getwd()"
#> [23] "   if (is.null(write_wd)){
#> [24] "     write_wd = 'inst/JAGS/'
#> [25] "   }"
#> [26] "   setwd(write_wd)"
#> [27] "   if (!manual_prior){
#> [28] "     if (distribution == \"Normal\"){
#> [29] "       JAGS_model = \"Emission_normal_JAGS.R\""
#> [30] "       cat(\"# Normal"
#> [31] "         model {"
#> [32] "           # priors"
#> [33] "             emission_sd ~ dunif(0.001 * sdY, 1000 * sdY)"
#> [34] "             emission_mean ~ dunif(0,maxY)"

```



```

#> [35] "          tau_em = 1 / (emission_sd^2)"
#> [36] ""
#> [37] "      #likelihood"
#> [38] "          for (i in 1:length(emission_xi)) {"
#> [39] "              emission_xi[i] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [40] "              pdf_obs[i] = dnorm(emission_xi[i], emission_mean, tau_em)"
#> [41] "          }"
#> [42] ""
#> [43] "      # derived quantities"
#> [44] "      # predict new emission tests"
#> [45] "          for (k in 1:n_draws){"
#> [46] "              emission_hat[k] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [47] "          }"
#> [48] "          for (h in 1:n_x_hat){"
#> [49] "              pdf_hat[h] = dnorm(x_hat[h], emission_mean, tau_em)"
#> [50] "          }"
#> [51] "      }\", file = JAGS_model)"
#> [52] "  } else if (distribution == \"Lognormal\"){"
#> [53] "      JAGS_model = \"Emission_lnorm_JAGS.R\""
#> [54] "      cat(\"# Lognormal"
#> [55] "      model {"
#> [56] "          # priors"
#> [57] "              sd_ln ~ dunif( 0.001 * sdOfLogY, 1000 * sdOfLogY )"
#> [58] "              tau_ln = 1 / (sd_ln^2)"
#> [59] "      "
#> [60] "          #this is in log-space, so it can be negative"
#> [61] "          u_ln ~ dnorm( meanOfLogY, 0.001 / sdOfLogY^2 )"
#> [62] "      "
#> [63] "      #likelihood"
#> [64] "          for (i in 1:length(emission_xi)){"
#> [65] "              emission_xi[i] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [66] "              pdf_obs[i] = dlnorm(emission_xi[i], u_ln, tau_ln)"
#> [67] "          }"
#> [68] "      "
#> [69] "      # derived quantities"
#> [70] "      # predict new emission tests"
#> [71] "          for (k in 1:n_draws){"
#> [72] "              emission_hat[k] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [73] "          }"
#> [74] "          for (h in 1:n_x_hat){"
#> [75] "              pdf_hat[h] = dlnorm(x_hat[h], u_ln, tau_ln)"
#> [76] "          }"
#> [77] "      }\", file = JAGS_model)"
#> [78] "  } else if (distribution == \"Skewed\"){"
#> [79] "      JAGS_model = \"Emission_skewed_JAGS.R\""
#> [80] "      cat(\"# Skewed"
#> [81] "      data {"
#> [82] "          for(i in 1:length(emission_xi)){"
#> [83] "              zeros[i] = 0"
#> [84] "          }"
#> [85] "      }"
#> [86] "      model {"
#> [87] "          # priors"
#> [88] "              omega ~ dunif(0, 100 * maxY) #must be positive"

```

```

#> [89] "          xi ~ dnorm(0, 1 / (100 * maxY))"
#> [90] "          alpha ~ dunif(-100, 100) # this is a very wide range"
#> [91] ""
#> [92] "      #likelihood"
#> [93] "          for (i in 1:length(emission_xi)) {"
#> [94] "              L[i] = ((2 / omega)"
#> [95] "                  * dnorm((emission_xi[i] - xi) / omega, 0, 1)"
#> [96] "                  * pnorm(alpha * (emission_xi[i] - xi) / omega, 0, 1))"
#> [97] "              zeros[i] ~ dpois(-log(L[i]) + 10000)"
#> [98] "          }"
#> [99] "      }\", file = JAGS_model)"
#> [100] "  } else if (distribution == 'Gamma'){
#> [101] "      JAGS_model = \"Emission_gamma_JAGS.R\"
#> [102] "      cat(\"# Gamma"
#> [103] "      model {"
#> [104] "          # priors"
#> [105] "              rate_em ~ dunif(0, maxY / (sdY^2)) #must be positive"
#> [106] "              shape_em ~ dunif(0, (maxY^2) / (sdY^2)) #must be positive"
#> [107] "      "
#> [108] "      #likelihood"
#> [109] "          for (i in 1:length(emission_xi)) {"
#> [110] "              emission_xi[i] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [111] "              pdf_obs[i] = dgamma(emission_xi[i], shape_em, rate_em)"
#> [112] "          }"
#> [113] "      "
#> [114] "      # derived quantities"
#> [115] "      # predict new emission tests"
#> [116] "          for (k in 1:n_draws){
#> [117] "              emission_hat[k] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [118] "          }"
#> [119] "          for (h in 1:n_x_hat){
#> [120] "              pdf_hat[h] = dgamma(x_hat[h], shape_em, rate_em)"
#> [121] "          }"
#> [122] "      }\", file = JAGS_model)"
#> [123] "  } else if (distribution == 'Beta'){
#> [124] "      JAGS_model = \"Emission_beta_JAGS.R\"
#> [125] "      cat(\"# Beta"
#> [126] "      model {"
#> [127] "          # priors"
#> [128] "              # note that both can be pos or neg, but the minimum is always -1"
#> [129] "              beta_em ~ dunif(-1, (0.25 - 1 + 0.25 / sdY^2 + 0.25^3 / sdY^2 - 2 *
#> [130] "                  0.25^2 / sdY^2) * 1.25)"
#> [131] "              alpha_em ~ dunif(-1, (0.75^2 / sdY^2 - 0.75^3 / sdY^2 - 0.75) * 1.25)"
#> [132] "      "
#> [133] "      #likelihood"
#> [134] "          for (i in 1:length(emission_xi)) {"
#> [135] "              emission_xi[i] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [136] "              pdf_obs[i] = dbeta(emission_xi[i], alpha_em, beta_em)"
#> [137] "          }"
#> [138] "      "
#> [139] "      # derived quantities"
#> [140] "      emission_mean = alpha_em / (alpha_em + beta_em)"
#> [141] "      # predict new emission tests"
#> [142] "          for (k in 1:n_draws){

```

```

#> [143] "          emission_hat[k] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [144] "      }"
#> [145] "      for (h in 1:n_x_hat){"
#> [146] "          pdf_hat[h] = dbeta(x_hat[h], alpha_em, beta_em)"
#> [147] "      }"
#> [148] "  }\", file = JAGS_model)"
#> [149] "  }"
#> [150] "  } else if (manual_prior){"
#> [151] "    if (distribution == \"Normal\"){"
#> [152] "      JAGS_model = \"Manual_emission_normal_JAGS.R\""
#> [153] "      cat(\"# Normal manual priors"
#> [154] "      model {"
#> [155] "        # priors"
#> [156] "          emission_sd ~ dunif(low1, up1)"
#> [157] "          emission_mean ~ dunif(low2, up2)"
#> [158] "          tau_em = 1 / (emission_sd^2)"
#> [159] "      }"
#> [160] "      #likelihood"
#> [161] "      for (i in 1:length(emission_xi)) {"
#> [162] "          emission_xi[i] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [163] "          pdf_obs[i] = dnorm(emission_xi[i], emission_mean, tau_em)"
#> [164] "      }"
#> [165] "      "
#> [166] "      # derived quantities"
#> [167] "      # predict new emission tests"
#> [168] "      for (k in 1:n_draws){"
#> [169] "          emission_hat[k] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [170] "      }"
#> [171] "      for (h in 1:n_x_hat){"
#> [172] "          pdf_hat[h] = dnorm(x_hat[h], emission_mean, tau_em)"
#> [173] "      }"
#> [174] "    }\", file = JAGS_model)"
#> [175] "  } else if (distribution == \"Lognormal\"){"
#> [176] "    JAGS_model = \"Manual_emission_lnorm_JAGS.R\""
#> [177] "    cat(\"# Lognormal manual priors"
#> [178] "    model {"
#> [179] "      # priors"
#> [180] "        sd_ln ~ dunif(low1, up1)"
#> [181] "        u_ln ~ dnorm(low2, up2) #this is in log-space, so it can be negative"
#> [182] "        tau_ln = 1 / (sd_ln^2)"
#> [183] "    }"
#> [184] "    #likelihood"
#> [185] "    for (i in 1:length(emission_xi)) {"
#> [186] "        emission_xi[i] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [187] "        pdf_obs[i] = dlnorm(emission_xi[i], u_ln, tau_ln)"
#> [188] "    }"
#> [189] "    "
#> [190] "    # derived quantities"
#> [191] "    # predict new emission tests"
#> [192] "    for (k in 1:n_draws){"
#> [193] "        emission_hat[k] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [194] "    }"
#> [195] "    for (h in 1:n_x_hat){"
#> [196] "        pdf_hat[h] = dlnorm(x_hat[h], u_ln, tau_ln)"

```

```

#> [197] "          }"
#> [198] "      }\", file = JAGS_model)"
#> [199] "  } else if (distribution == \"Skewed\"){"
#> [200] "    JAGS_model = \"Manual_emission_skewed_JAGS.R\""
#> [201] "    cat(\"# Skewed manual priors"
#> [202] "      data {"
#> [203] "          for(i in 1:length(emission_xi)) {"
#> [204] "              zeros[i] = 0"
#> [205] "          }"
#> [206] "      }"
#> [207] "      model {"
#> [208] "      # priors"
#> [209] "          omega ~ dunif(low1, up1) #must be positive"
#> [210] "          xi ~ dunif(low2, up2)"
#> [211] "          alpha ~ dunif(low3, up3) # this is a very wide range"
#> [212] ""
#> [213] "      #likelihood"
#> [214] "          for (i in 1:length(emission_xi)) {"
#> [215] "              L[i] = ((2 / omega)"
#> [216] "                  * dnorm((emission_xi[i] - xi) / omega, 0, 1)"
#> [217] "                  * pnorm(alpha * (emission_xi[i] - xi) / omega, 0, 1))"
#> [218] "              zeros[i] ~ dpois(-log(L[i]) + 10000)"
#> [219] "          }"
#> [220] "      }\", file = JAGS_model)"
#> [221] "  } else if (distribution == 'Gamma'){
#> [222] "    JAGS_model = \"Manual_emission_gamma_JAGS.R\""
#> [223] "    cat(\"# Gamma manual priors"
#> [224] "      model {"
#> [225] "      # priors"
#> [226] "          rate_em ~ dunif(low1, up1) #must be positive"
#> [227] "          shape_em ~ dunif(low2, up2) #must be positive"
#> [228] ""
#> [229] "      #likelihood"
#> [230] "          for (i in 1:length(emission_xi)) {"
#> [231] "              emission_xi[i] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [232] "              pdf_obs[i] = dgamma(emission_xi[i], shape_em, rate_em)"
#> [233] "          }"
#> [234] ""
#> [235] "      # derived quantities"
#> [236] "      # predict new emission tests"
#> [237] "          for (k in 1:n_draws){
#> [238] "              emission_hat[k] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [239] "          }"
#> [240] "          for (h in 1:n_x_hat){
#> [241] "              pdf_hat[h] = dgamma(x_hat[h], shape_em, rate_em)"
#> [242] "          }"
#> [243] "      }\", file = JAGS_model)"
#> [244] "  } else if (distribution == 'Beta'){
#> [245] "    JAGS_model = \"Manual_emission_beta_JAGS.R\""
#> [246] "    cat(\"# Beta manual priors"
#> [247] "      model {"
#> [248] "      # priors"
#> [249] "          # note that both can be pos or neg, but the minimum is always -1"
#> [250] "          alpha_em ~ dunif(low1, up1)"

```

```

#> [251] "          beta_em ~ dunif(low2, up2)"
#> [252] ""
#> [253] "      #likelihood"
#> [254] "          for (i in 1:length(emission_xi)) {"
#> [255] "              emission_xi[i] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [256] "              pdf_obs[i] = dbeta(emission_xi[i], alpha_em, beta_em)"
#> [257] "          }"
#> [258] ""
#> [259] "      # derived quantities"
#> [260] "          emission_mean = alpha_em / (alpha_em + beta_em)"
#> [261] "      # predict new emission tests"
#> [262] "          for (k in 1:n_draws){"
#> [263] "              emission_hat[k] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [264] "          }"
#> [265] "          for (h in 1:n_x_hat){"
#> [266] "              pdf_hat[h] = dbeta(x_hat[h], alpha_em, beta_em)"
#> [267] "          }"
#> [268] "      }\n", file = JAGS_model)"
#> [269] "  }"
#> [270] "}"
#> [271] "  setwd(current_wd)"
#> [272] "}"

```

setup_likelihood()

```

#> function (distribution, data, emissions, manual_prior = FALSE,
#>   prior_list = NULL, random = FALSE, RNG.state = NULL, custom_model = NULL,
#>   custom_params = NULL, custom_init = NULL)
#> {
#>   JAGS_path = system.file("JAGS", package = "UPLforOAR", mustWork = TRUE)
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   if (any(is.na(data_temp$emissions))) {
#>     warning("Emissions with NA values have been removed")
#>     data_temp = stats::na.omit(data_temp)
#>   }
#>   if (!is.numeric(data_temp$emissions)) {
#>     stop("Emissions must be numeric")
#>   }
#>   mu = mean(data_temp$emissions)
#>   sigma = stats::sd(data_temp$emissions)
#>   if (sigma == 0) {
#>     stop("Cannot calculate UPL with zero variance data")
#>   }
#>   if (is.null(RNG.state)) {
#>     RNG.state = list(list(.RNG.name = "base::Wichmann-Hill",
#>       .RNG.seed = 5), list(.RNG.name = "base::Marsaglia-Multicarry",
#>       .RNG.seed = 12), list(.RNG.name = "base::Super-Duper",
#>       .RNG.seed = 151))
#>   }
#>   if (!manual_prior) {
#>     if (distribution == "Custom") {
#>       JAGS_model = runjags::read.jagsfile(custom_model)
#>       par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>         custom_params)

```

```

#>         data_inits_temp = list(custom_init[[1]], custom_init[[2]],
#>                                custom_init[[3]])
#>     }
#>     if (distribution == "Normal") {
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Emission_normal_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "emission_mean", "emission_sd")
#>         data_inits_temp = list(list(emission_mean = mu, emission_sd = sigma),
#>             list(emission_mean = 1.5 * mu, emission_sd = 0.5 *
#>                 sigma), list(emission_mean = 0.5 * mu, emission_sd = 1.5 *
#>                 sigma))
#>     }
#>     else if (distribution == "Lognormal") {
#>         ln_emiss = log(data_temp$emissions)
#>         ln_mu = mean(ln_emiss, na.rm = TRUE)
#>         ln_sig = stats::sd(ln_emiss, na.rm = TRUE)
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Emission_lnorm_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "u_ln", "sd_ln")
#>         data_inits_temp = list(list(u_ln = ln_mu, sd_ln = ln_sig),
#>             list(u_ln = 1.5 * ln_mu, sd_ln = 0.5 * ln_sig),
#>             list(u_ln = 0.5 * ln_mu, sd_ln = 1.5 * ln_sig))
#>     }
#>     else if (distribution == "Skewed") {
#>         skew1 = min(0.99, abs((1/length(data_temp$emissions)) *
#>             sum(((data_temp$emissions - mu)/sigma)^3)))
#>         delta = sqrt((pi/2) * ((abs(skew1)^(2/3))/((abs(skew1)^(2/3)) +
#>             ((4 - pi)/2)^(2/3))))
#>         delta = delta * abs(skew1)/skew1
#>         alpha = delta/sqrt(1 - delta^2)
#>         omega = sigma/sqrt(1 - 2 * delta^2/pi)
#>         xi = mu - omega * delta * sqrt(2/pi)
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Emission_skewed_JAGS.R"))
#>         par_list = c("omega", "xi", "alpha")
#>         data_inits_temp = list(list(xi = xi, omega = omega,
#>             alpha = alpha), list(xi = 1.5 * xi, omega = 0.5 *
#>                 omega, alpha = 0.5 * alpha), list(xi = 0.5 *
#>                 xi, omega = 1.5 * omega, alpha = 1.5 * alpha))
#>     }
#>     else if (distribution == "Gamma") {
#>         shape = mu^2/sigma^2
#>         rate = mu/sigma^2
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Emission_gamma_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "rate_em", "shape_em")
#>         data_inits_temp = list(list(rate_em = rate, shape_em = shape),
#>             list(rate_em = 1.5 * rate, shape_em = 0.5 * shape),
#>             list(rate_em = 0.5 * rate, shape_em = 1.5 * shape))
#>     }
#>     else if (distribution == "Beta") {

```

```

#>         if (min(data_temp$emissions) < 0) {
#>             stop("Cannot use beta distribution with emissions less than 0")
#>         }
#>         if (max(data_temp$emissions) > 1) {
#>             stop("Cannot use beta distribution with emissions greater than 1")
#>         }
#>         alpha = mu^2/sigma^2 - mu^3/sigma^2 - mu
#>         beta = mu - 1 + mu/sigma^2 + mu^3/sigma^2 - 2 * mu^2/sigma^2
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Emission_beta_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "alpha_em", "beta_em")
#>         data_inits_temp = list(list(beta_em = beta, alpha_em = alpha),
#>             list(beta_em = 1.5 * beta, alpha_em = 0.5 * alpha),
#>             list(beta_em = 0.5 * beta, alpha_em = 1.5 * alpha))
#>     }
#>     if (random) {
#>         new_state = rjags::parallel.seeds("base::BaseRNG",
#>             3)
#>     }
#>     else if (!random) {
#>         new_state = RNG.state
#>     }
#>     data_inits = list(c(data_inits_temp[[1]], new_state[[1]]),
#>         c(data_inits_temp[[2]], new_state[[2]]), c(data_inits_temp[[3]],
#>             new_state[[3]]))
#>     output = list(model_code = JAGS_model, par_list = par_list,
#>         data = data_temp, distribution = distribution, dat_inits = data_inits,
#>         manual_prior = manual_prior, state = new_state)
#> }
#> else if (manual_prior) {
#>     if (is.null(prior_list)) {
#>         stop("Please provide a list of upper and lower bounds for manual priors")
#>     }
#>     if (distribution == "Normal") {
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Manual_emission_normal_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "emission_mean", "emission_sd")
#>         if (any(prior_list[1:2] <= 0)) {
#>             stop("Prior limits must be positive")
#>         }
#>         data_inits_temp = list(list(emission_mean = mean(c(prior_list[3],
#>             prior_list[4])), emission_sd = mean(c(prior_list[1],
#>             prior_list[2]))), list(emission_mean = 0.9 *
#>             prior_list[4], emission_sd = 0.9 * prior_list[2]),
#>             list(emission_mean = 1.1 * prior_list[3], emission_sd = 1.1 *
#>                 prior_list[1]))
#>     }
#>     else if (distribution == "Lognormal") {
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Manual_emission_lnorm_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "u_ln", "sd_ln")

```

```

#>         if (any(prior_list[1:2] <= 0)) {
#>             stop("Prior limits must be positive")
#>         }
#>         if (prior_list[4] < 0) {
#>             initial1 = 1.1 * prior_list[4]
#>         }
#>         else if (prior_list[4] > 0) {
#>             initial1 = 0.9 * prior_list[4]
#>         }
#>         if (prior_list[3] < 0) {
#>             initial2 = 0.9 * prior_list[3]
#>         }
#>         else if (prior_list[3] > 0) {
#>             initial2 = 1.1 * prior_list[3]
#>         }
#>         data_inits_temp = list(list(u_ln = mean(c(prior_list[3],
#>             prior_list[4])), sd_ln = mean(c(prior_list[1],
#>             prior_list[2]))), list(u_ln = initial1, sd_ln = 0.9 *
#>             prior_list[2]), list(u_ln = initial2, sd_ln = 1.1 *
#>             prior_list[1]))
#>     }
#>     else if (distribution == "Skewed") {
#>         if (any(prior_list[1:2] <= 0)) {
#>             stop("Prior limits must be positive")
#>         }
#>         if (prior_list[4] < 0) {
#>             initial1 = 1.1 * prior_list[4]
#>         }
#>         else if (prior_list[4] > 0) {
#>             initial1 = 0.9 * prior_list[4]
#>         }
#>         if (prior_list[3] < 0) {
#>             initial2 = 0.9 * prior_list[3]
#>         }
#>         else if (prior_list[3] > 0) {
#>             initial2 = 1.1 * prior_list[3]
#>         }
#>         if (prior_list[6] < 0) {
#>             initial3 = 1.1 * prior_list[6]
#>         }
#>         else if (prior_list[6] > 0) {
#>             initial3 = 0.9 * prior_list[6]
#>         }
#>         if (prior_list[5] < 0) {
#>             initial4 = 0.9 * prior_list[5]
#>         }
#>         else if (prior_list[5] > 0) {
#>             initial4 = 1.1 * prior_list[5]
#>         }
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Manual_emission_skewed_JAGS.R"))
#>         par_list = c("omega", "xi", "alpha")
#>         data_inits_temp = list(list(xi = mean(c(prior_list[3],
#>             prior_list[4])), omega = mean(c(prior_list[1],

```



```

#>         prior_list[2])), alpha = mean(c(prior_list[5],
#>         prior_list[6])), list(xi = initial1, omega = 1.1 *
#>         prior_list[1], alpha = initial4), list(xi = initial2,
#>         omega = 0.9 * prior_list[2], alpha = initial3))
#>     }
#>     else if (distribution == "Gamma") {
#>         if (any(prior_list <= 0)) {
#>             stop("Prior limits must be positive")
#>         }
#>         JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>             "/Manual_emission_gamma_JAGS.R"))
#>         par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>             "rate_em", "shape_em")
#>         data_inits_temp = list(list(rate_em = mean(c(prior_list[1],
#>             prior_list[2])), shape_em = mean(c(prior_list[3],
#>             prior_list[4])), list(rate_em = 1.1 * prior_list[1],
#>             shape_em = 1.1 * prior_list[3]), list(rate_em = 0.9 *
#>             prior_list[2], shape_em = 0.9 * prior_list[4]))
#>     }
#>     else if (distribution == "Beta") {
#>         if (min(data_temp$emissions) < 0) {
#>             stop("Cannot use beta distribution with emissions less than 0")
#>         }
#>         if (max(data_temp$emissions) > 1) {
#>             stop("Cannot use beta distribution with emissions greater than 1")
#>         }
#>         if (prior_list[2] < 0) {
#>             initial1 = 1.1 * prior_list[2]
#>         }
#>         else if (prior_list[2] > 0) {
#>             initial1 = 0.9 * prior_list[2]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[1] < 0) {
#>             initial2 = 0.9 * prior_list[1]
#>         }
#>         else if (prior_list[1] > 0) {
#>             initial2 = 1.1 * prior_list[1]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[4] < 0) {
#>             initial3 = 1.1 * prior_list[4]
#>         }
#>         else if (prior_list[4] > 0) {
#>             initial3 = 0.9 * prior_list[4]
#>             if (initial1 < (-1)) {
#>                 initial1 = -0.99999
#>             }
#>         }
#>         if (prior_list[3] < 0) {

```

```

#>         initial4 = 0.9 * prior_list[3]
#>     }
#>     else if (prior_list[3] > 0) {
#>         initial4 = 1.1 * prior_list[3]
#>         if (initial1 < (-1)) {
#>             initial1 = -0.99999
#>         }
#>     }
#>     JAGS_model = runjags::read.jagsfile(paste0(JAGS_path,
#>         "/Manual_emission_beta_JAGS.R"))
#>     par_list = c("emission_hat", "pdf_obs", "pdf_hat",
#>         "alpha_em", "beta_em")
#>     data_inits_temp = list(list(beta_em = mean(c(prior_list[3],
#>         prior_list[4])), alpha_em = mean(c(prior_list[1],
#>         prior_list[2]))), list(beta_em = initial3, alpha_em = initial1),
#>         list(beta_em = initial4, alpha_em = initial2))
#> }
#> if (random) {
#>     new_state = rjags::parallel.seeds("base::BaseRNG",
#>         3)
#> }
#> else if (!random) {
#>     new_state = RNG.state
#> }
#> data_inits = list(c(data_inits_temp[[1]], new_state[[1]]),
#>     c(data_inits_temp[[2]], new_state[[2]]), c(data_inits_temp[[3]],
#>     new_state[[3]]))
#> output = list(model_code = JAGS_model, par_list = par_list,
#>     data = data_temp, manual_prior = manual_prior, distribution = distribution,
#>     dat_inits = data_inits, prior_list = prior_list,
#>     state = new_state)
#> }
#> return(output)
#> }
#> <bytecode: 0x0000028305894a58>
#> <environment: namespace:UPLforOAR>

```

run_likelihood()

```

#> function (model_input, xvals = NULL, minY = 0, maxY = NULL, future_runs = 3)
#> {
#>     future_runs = as.integer(future_runs)
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     manual_prior = model_input$manual_prior
#>     data = model_input$data
#>     Sys.setenv(`_R_CHECK_LIMIT_CORES_` = FALSE)
#>     verify_install = runjags::testjags(silent = TRUE)
#>     if (!verify_install$JAGS.available) {
#>         stop("Please install JAGS")
#>     }
#> }

```

```

#>   }
#>   if (is.null(maxY)) {
#>     maxY = 3 * max(data$emissions)
#>   }
#>   if ((model_input$distribution == "Beta") & (maxY > 1)) {
#>     stop("Cannot use beta distribution with max emissions greater than 1")
#>   }
#>   n.adapt = 10000
#>   n.update = 10000
#>   n.iter = 10000
#>   if (is.null(xvals)) {
#>     xvals = seq(minY, maxY, length.out = 1024)
#>   }
#>   if ((model_input$distribution == "Beta") & (max(xvals) >
#>     1)) {
#>     stop("Cannot use beta distribution with max xvals greater than 1")
#>   }
#>   if (!manual_prior) {
#>     data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>       minY = minY, sdOfLogY = stats::sd(log(data$emissions),
#>         na.rm = T), maxY = maxY, sdY = stats::sd(data$emissions),
#>       meanOfLogY = mean(log(data$emissions), na.rm = T),
#>       pi = pi, x_hat = xvals, n_x_hat = length(xvals))
#>   }
#>   else if (manual_prior) {
#>     if (length(model_input$prior_list) == 4) {
#>       data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>         minY = minY, sdOfLogY = stats::sd(log(data$emissions),
#>           na.rm = T), maxY = maxY, sdY = stats::sd(data$emissions),
#>         meanOfLogY = mean(log(data$emissions), na.rm = T),
#>         pi = pi, x_hat = xvals, n_x_hat = length(xvals),
#>         low1 = model_input$prior_list[1], up1 = model_input$prior_list[2],
#>         low2 = model_input$prior_list[3], up2 = model_input$prior_list[4])
#>     }
#>     else if (length(model_input$prior_list) == 6) {
#>       data_list = list(emission_xi = data$emissions, n_draws = future_runs,
#>         minY = minY, sdOfLogY = stats::sd(log(data$emissions),
#>           na.rm = T), maxY = maxY, sdY = stats::sd(data$emissions),
#>         meanOfLogY = mean(log(data$emissions), na.rm = T),
#>         pi = pi, x_hat = xvals, n_x_hat = length(xvals),
#>         low1 = model_input$prior_list[1], up1 = model_input$prior_list[2],
#>         low2 = model_input$prior_list[3], up2 = model_input$prior_list[4],
#>         low3 = model_input$prior_list[5], up3 = model_input$prior_list[6])
#>     }
#>   }
#>   cl3 = parallel::makeCluster(3)
#>   rjm = suppressWarnings(runjags::run.jags(model = model_input$model_code,
#>     data = data_list, monitor = model_input$par_list, method = "rjparallel",
#>     summarise = FALSE, cl = cl3, n.chains = length(model_input$dat_inits),
#>     burnin = n.update, adapt = n.adapt, sample = n.iter,
#>     inits = model_input$dat_inits))
#>   output = list(run_results = rjm, distribution = model_input$distribution,
#>     manual_prior = manual_prior, maxY = maxY, minY = minY,
#>     data = model_input$data, xvals = xvals, future_runs = future_runs,

```

```

#>         state = model_input$state)
#>     parallel::stopCluster(cl3)
#>     return(output)
#> }
#> <bytecode: 0x0000028305f14e10>
#> <environment: namespace:UPLforOAR>

```

output_likelihood()

```

#> function (jags_model_run, significance = 0.99)
#> {
#>     if (significance >= 1) {
#>         stop("significance must be greater than 0 and less than 1")
#>     }
#>     if (significance <= 0) {
#>         stop("significance must be greater than 0 and less than 1")
#>     }
#>     distribution = jags_model_run$distribution
#>     data = jags_model_run$data
#>     future_runs = jags_model_run$future_runs
#>     xvals = jags_model_run$xvals
#>     minY = jags_model_run$minY
#>     maxY = jags_model_run$maxY
#>     if (distribution == "Skewed") {
#>         xi_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>             vars = "xi")))
#>         omega_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>             vars = "omega")))
#>         alpha_quant = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>             vars = "alpha")))
#>         hat_quant = matrix(nrow = length(xi_quant), ncol = future_runs,
#>             data = NA)
#>         pdf_obs = matrix(ncol = nrow(data), nrow = length(xi_quant),
#>             data = NA)
#>         pdf_hat = matrix(ncol = length(xvals), nrow = length(xi_quant),
#>             data = NA)
#>         for (i in 1:length(xi_quant)) {
#>             set.seed(12)
#>             Fy_sn = sn::dsn(xvals, xi = (xi_quant[i]), omega = (omega_quant[i]),
#>                 alpha = (alpha_quant[i]))
#>             set.seed(12)
#>             pdf_obs[i, ] = sn::dsn(data$emissions, xi = (xi_quant[i]),
#>                 omega = (omega_quant[i]), alpha = (alpha_quant[i]))
#>             pdf_hat[i, ] = Fy_sn
#>             if (all(Fy_sn == 0)) {
#>                 for (k in 1:future_runs) {
#>                     hat_quant[i, k] = NA
#>                 }
#>             }
#>             else {
#>                 for (k in 1:future_runs) {
#>                     set.seed(12)
#>                     hat_quant[i, k] = sample(x = xvals, size = 1,
#>                         prob = Fy_sn, replace = T)
#>                 }
#>             }
#>         }
#>     }
#> }

```

```

#>         }
#>     }
#> }
#>     hat_quant = tibble::as_tibble(hat_quant, .name_repair = "minimal")
#> }
#> else {
#>     pdf_obs = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "pdf_obs")))
#>     hat_quant = tibble::as_tibble(as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "emission_hat"))))
#>     pdf_hat = as.matrix(runjags::combine.mcmc(coda::as.mcmc.list(jags_model_run$run_results,
#>         vars = "pdf_hat")))
#> }
#> names(hat_quant) = sprintf("run%s", seq(1:future_runs))
#> run3_mean = rowMeans(hat_quant)
#> pred_99_3rep = stats::quantile(as.matrix(stats::na.omit(run3_mean)),
#>     probs = c(significance))
#> pdf_hat_quant = matrixStats::colQuantiles(pdf_hat, probs = c(0.5))
#> density_hat = tibble::tibble(pdf_hat = pdf_hat_quant, x_hat = xvals)
#> density_hat = subset(density_hat, is.finite(density_hat$pdf_hat))
#> pdf_obs_quant = tibble::as_tibble(matrixStats::colQuantiles(pdf_obs,
#>     probs = c(0.025, 0.5, 0.975)), .name_repair = "minimal")
#> names(pdf_obs_quant) = c("low", "med", "up")
#> pdf_obs_quant$emissions = data$emissions
#> density_hat$distr = rep(distribution, nrow(density_hat))
#> pdf_obs_quant$distr = rep(distribution, nrow(pdf_obs_quant))
#> pred_mean = mean(hat_quant$run1, na.rm = TRUE)
#> output = list(predicted_mean = pred_mean, UPL_Bayes = pred_99_3rep,
#>     obs_pdf = pdf_obs_quant, pred_pdf = density_hat, distr = distribution,
#>     minY = minY, maxY = maxY, state = jags_model_run$state)
#>     return(output)
#> }
#> <bytecode: 0x00000283077bc5e0>
#> <environment: namespace:UPLforOAR>

```

converge_likelihood()

```

#> function (jags_model_run, custom_params = NULL)
#> {
#>     distribution = jags_model_run$distribution
#>     gelman_list = c()
#>     convYN = c()
#>     if (distribution == "Skewed") {
#>         params_list = c("omega", "xi", "alpha")
#>     }
#>     if (distribution == "Normal") {
#>         params_list = c("emission_mean", "emission_sd")
#>     }
#>     if (distribution == "Lognormal") {
#>         params_list = c("u_ln", "sd_ln")
#>     }
#>     if (distribution == "Gamma") {
#>         params_list = c("rate_em", "shape_em")
#>     }
#> }

```

```

#>   if (distribution == "Beta") {
#>     params_list = c("alpha_em", "beta_em")
#>   }
#>   if (distribution == "Custom") {
#>     params_list = custom_params
#>   }
#>   for (i in 1:length(params_list)) {
#>     param = params_list[i]
#>     result = coda::gelman.diag(coda::as.mcmc.list(jags_model_run$run_results,
#>       vars = param))
#>     gelman_list[i] = result$psrf[1]
#>     if (result$psrf[1] > 1.2) {
#>       convYN[i] = "No"
#>     }
#>     else if (result$psrf[1] < 1.1) {
#>       convYN[i] = "Yes"
#>     }
#>     else if ((result$psrf[1] > 1.1) & (result$psrf[1] < 1.2)) {
#>       convYN[i] = "Weak convergence"
#>     }
#>   }
#>   results = tibble::tibble(params = params_list, gelman_diag = gelman_list,
#>     convYN = convYN)
#>   distr_names = tibble::tibble(distr = rep(distribution, nrow(results)))
#>   results = cbind(distr_names, results)
#>   return(results)
#> }
#> <bytecode: 0x0000028307605300>
#> <environment: namespace:UPLforOAR>

```

converge_figs()

```

#> function (distribution, jags_model_run, custom_params = NULL)
#> {
#>   if (distribution == "Normal") {
#>     par_list = c("emission_mean", "emission_sd")
#>   }
#>   else if (distribution == "Lognormal") {
#>     par_list = c("u_ln", "sd_ln")
#>   }
#>   else if (distribution == "Skewed") {
#>     par_list = c("omega", "xi", "alpha")
#>   }
#>   else if (distribution == "Gamma") {
#>     par_list = c("rate_em", "shape_em")
#>   }
#>   else if (distribution == "Beta") {
#>     par_list = c("alpha_em", "beta_em")
#>   }
#>   else if (distribution == "Custom") {
#>     par_list = custom_params
#>   }
#>   fig_list = list()
#>   for (i in 1:length(par_list)) local({

```

```

#> i = i
#> step3 = coda::as.mcmc.list(jags_model_run$run_results,
#>   vars = par_list[i])
#> step4 = runjags::combine.mcmc(step3)
#> p1 = ggplot2::ggplot() + ggplot2::geom_line(ggplot2::aes(color = "a",
#>   x = 1:10000, y = as.vector(step3[[1]])), linewidth = 0.5,
#>   linetype = 1) + ggplot2::geom_line(ggplot2::aes(color = "b",
#>   x = 1:10000, y = as.vector(step3[[2]])), linewidth = 0.5,
#>   linetype = 2) + ggplot2::geom_line(ggplot2::aes(color = "c",
#>   x = 1:10000, y = as.vector(step3[[3]])), linewidth = 0.5,
#>   linetype = 3) + mcmc_theme() + ggplot2::xlab("Iterations") +
#>   ggplot2::ylab(par_list[i])
#> p2 = ggplot2::ggplot() + ggplot2::geom_histogram(ggplot2::aes(step4),
#>   color = "black", fill = "grey") + ggplot2::xlab(par_list[i]) +
#>   mcmc_theme()
#> p12 = p1 + p2 + patchwork::plot_layout(ncol = 2) + patchwork::plot_annotation(title = distrib,
#>   ggplot2::theme(plot.title = ggplot2::element_text(size = 20,
#>     hjust = 0.5))
#>   fig_list[[i]] <- p12
#> })
#> return(fig_list)
#> }
#> <bytecode: 0x00000283366af418>
#> <environment: namespace:UPLforOAR>

```

fit_likelihood()

```

#> function (likelihood_result, up = Inf, low = 0, kernel = "gamma",
#>   bw = NULL)
#> {
#>   obs_pdf_temp = likelihood_result$obs_pdf
#>   pred_pdf_temp = likelihood_result$pred_pdf
#>   minY = likelihood_result$minY
#>   maxY = likelihood_result$maxY
#>   if (up < maxY) {
#>     up = maxY
#>   }
#>   if (low > minY) {
#>     low = minY
#>   }
#>   obs_dens_results = obs_density(data = obs_pdf_temp, emissions = "emissions",
#>     up = up, low = low, bw = bw, kernel = kernel, xvals = pred_pdf_temp$x_hat)
#>   Obs_onPoint = obs_dens_results$Obs_onPoint
#>   obs_pdf_dat = dplyr::left_join(Obs_onPoint, obs_pdf_temp,
#>     by = "emissions", multiple = "any")
#>   obs_pdf_dat$inCI = NA
#>   for (k in 1:nrow(obs_pdf_dat)) {
#>     if ((obs_pdf_dat$ydens[k] > obs_pdf_dat$low[k]) & (obs_pdf_dat$ydens[k] <
#>       obs_pdf_dat$up[k])) {
#>       obs_pdf_dat$inCI[k] = 1
#>     }
#>     else {
#>       obs_pdf_dat$inCI[k] = 0
#>     }
#>   }
#> }

```

```

#>   }
#>   obs_den_df = obs_dens_results$obs_den_df
#>   xhat_pdf_dat = dplyr::full_join(pred_pdf_temp, obs_den_df,
#>     by = "x_hat")
#>   SSE = (sum((obs_pdf_dat$ydens - obs_pdf_dat$med)^2))
#>   pdf_integral = sfsmisc::integrate.xy(pred_pdf_temp$x_hat,
#>     pred_pdf_temp$pdf_hat)
#>   fit_temp = list(distr = likelihood_result$distr, pdf_integral = pdf_integral,
#>     SSE = SSE, good_vals = sum(obs_pdf_dat$inCI), obs_pdf_dat = obs_pdf_dat,
#>     xhat_pdf_dat = xhat_pdf_dat, UPL_Bayes = likelihood_result$UPL_Bayes,
#>     state = likelihood_result$state)
#>   return(fit_temp)
#> }
#> <bytecode: 0x000002830ce93820>
#> <environment: namespace:UPLforOAR>

```

Bayesian_UPL()

```

#> function (distr_list = c("Normal", "Skewed", "Lognormal", "Gamma",
#>   "Beta"), data, emissions, future_runs = 3, significance = 0.99,
#>   xvals = NULL, maxY = NULL, minY = 0, RNG.state = NULL, up = Inf,
#>   low = 0, kernel = "gamma", bw = NULL, convergence_report = FALSE,
#>   random = FALSE, manual_prior = FALSE, prior_list = NULL)
#> {
#>   if (convergence_report == TRUE) {
#>     figs_list = list()
#>   }
#>   mod_output_list = c()
#>   conv_output = tibble::tibble()
#>   if (manual_prior) {
#>     if (length(distr_list) > 1) {
#>       stop("You can only run one distribution at a time if supplying priors manually")
#>     }
#>     distribution = distr_list[1]
#>     mod_bayes = setup_likelihood(distribution = distribution,
#>       data = data, emissions = emissions, RNG.state = RNG.state,
#>       manual_prior = manual_prior, random = random, prior_list = prior_list)
#>     mod_run = run_likelihood(model_input = mod_bayes, maxY = maxY,
#>       minY = minY, future_runs = future_runs, xvals = xvals)
#>     manual_priors = mod_bayes$manual_priors
#>     mod_output = output_likelihood(jags_model_run = mod_run,
#>       significance = significance)
#>     mod_fit = fit_likelihood(likelihood_result = mod_output,
#>       up = up, low = low, kernel = kernel, bw = bw)
#>     mod_output_list[[1]] = mod_fit
#>     mod_converge = converge_likelihood(mod_run)
#>     conv_output = rbind(conv_output, mod_converge)
#>     if (convergence_report == TRUE) {
#>       fig_set = converge_figs(distribution, mod_run)
#>       figs_list[[1]] = fig_set
#>       rm(fig_set)
#>     }
#>     rm(mod_run, mod_output, mod_fit)
#>     gc()

```



```

#>   }
#>   if (!manual_prior) {
#>     for (j in 1:length(distr_list)) {
#>       distribution = distr_list[j]
#>       if (random == TRUE) {
#>         if (j == 1) {
#>           random = TRUE
#>         }
#>         else if (j > 1) {
#>           random = FALSE
#>           RNG.state = mod_output_list[[j - 1]]$state
#>         }
#>       }
#>       mod_bayes = setup_likelihood(distribution = distribution,
#>                                   data = data, emissions = emissions, RNG.state = RNG.state,
#>                                   manual_prior = FALSE, random = random)
#>       mod_run = run_likelihood(model_input = mod_bayes,
#>                               maxY = maxY, minY = minY, future_runs = future_runs,
#>                               xvals = xvals)
#>       mod_output = output_likelihood(jags_model_run = mod_run,
#>                                     significance = significance)
#>       mod_fit = fit_likelihood(likelihood_result = mod_output,
#>                               up = up, low = low, kernel = kernel, bw = bw)
#>       mod_output_list[[j]] = mod_fit
#>       mod_converge = converge_likelihood(mod_run)
#>       conv_output = rbind(conv_output, mod_converge)
#>       if (convergence_report == TRUE) {
#>         fig_set = converge_figs(distribution, mod_run)
#>         figs_list[[j]] = fig_set
#>         rm(fig_set)
#>       }
#>       rm(mod_run, mod_output, mod_fit)
#>       gc()
#>     }
#>   }
#>   if (convergence_report == TRUE) {
#>     current_wd = getwd()
#>     template_path = system.file("templates", package = "UPLforOAR",
#>                                mustWork = TRUE)
#>     rmarkdown::render(paste0(template_path, "/convergence_template.Rmd"),
#>                       output_dir = current_wd, output_file = paste0("Bayesian_UPL_convergence_",
#>                             format(Sys.time(), "%m%d%Y-%H%M"))
#>   }
#>   fit_table = tibble::tibble(distr = unlist(lapply(mod_output_list,
#>     "[[", "distr")), UPL = (as.numeric(lapply(mod_output_list,
#>     "[[", "UPL_Bayes"))), SSE = (as.numeric(lapply(mod_output_list,
#>     "[[", "SSE"))), Obs_in_CI = (as.numeric(lapply(mod_output_list,
#>     "[[", "good_vals"))), pdf_integral = (as.numeric(lapply(mod_output_list,
#>     "[[", "pdf_integral"))))
#>   state = mod_output_list[[length(distr_list)]]$state
#>   obs_pdf_dat = tibble::tibble()
#>   for (i in 1:length(distr_list)) {
#>     obs_temp = mod_output_list[[i]]$obs_pdf_dat
#>     obs_pdf_dat = rbind(obs_pdf_dat, obs_temp)

```

```

#>   }
#>   pred_pdf_dat = tibble::tibble()
#>   for (i in 1:length(distr_list)) {
#>     pred_temp = mod_output_list[[i]]$xhat_pdf_dat
#>     pred_pdf_dat = rbind(pred_pdf_dat, pred_temp)
#>   }
#>   if (any(conv_output$convYN != "Yes")) {
#>     warning("Some parameters have not converged")
#>   }
#>   return_list = list(fit_table = fit_table, conv_output = conv_output,
#>     obs_pdf_dat = obs_pdf_dat, pred_pdf_dat = pred_pdf_dat,
#>     state = state)
#>   return(return_list)
#> }
#> <bytecode: 0x000002830587c200>
#> <environment: namespace:UPLforOAR>

```

obs_density()

```

#> function (data, emissions, xvals = NULL, up = Inf, low = 0, kernel = "gamma",
#>   bw = NULL)
#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   sigma = stats::sd(data_temp$emissions)
#>   if (sigma == 0) {
#>     stop("Cannot find density for zero variance data")
#>   }
#>   if (is.null(bw)) {
#>     bw = sigma * nrow(data_temp)^(-2/5)
#>   }
#>   if (is.null(xvals)) {
#>     xvals = seq(0, 3 * max(data_temp$emissions), length.out = 1024)
#>   }
#>   if (is.numeric(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = data_temp$emissions, a = low, b = up, proper = TRUE,
#>       kertype = kernel, h = bw)
#>     obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>       h = bw)
#>   }
#>   else if (is.character(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = data_temp$emissions, bwmethod = bw, a = low,
#>       b = up, proper = TRUE, kertype = kernel)
#>     obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>       bwmethod = bw)
#>   }
#>   obs_den_df = tibble::tibble(x_hat = xvals, ydens = obs_den_df$f)
#>   Obs_onPoint = tibble::tibble(emissions = data_temp$emissions,
#>     ydens = Obs_onPoint$f)
#>   test_int = sfsmisc::integrate.xy(obs_den_df$x_hat, obs_den_df$ydens)
#>   if (abs(test_int - 1) > 0.05) {

```

```

#>      warning("density distribution does not integrate to 1,\n      consider adjusting xvals
#>    }
#>    output = list(Obs_onPoint = Obs_onPoint, obs_den_df = obs_den_df)
#>    return(output)
#>  }
#> <bytecode: 0x000002830ce03120>
#> <environment: namespace:UPLforOAR>

```

make_environment()

```

#> function (data, HAP, emissions, sources, subcat_name = NA, subcat_level = NA,
#>   type = c("New", "Existing", "As-is"), CAA_section = 112,
#>   national_N = NA, meas_unit, future_runs = 3, significance = 0.99,
#>   ...)
#> {
#>   future_runs = as.integer(future_runs)
#>   if (!is.integer(future_runs)) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (future_runs < 1) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (significance >= 1) {
#>     stop("significance must be greater then 0 and less than 1")
#>   }
#>   if (significance <= 0) {
#>     stop("significance must be greater then 0 and less than 1")
#>   }
#>   if (is.na(subcat_name)) {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>       sources = data[[sources]])
#>   }
#>   else {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>       sources = data[[sources]], subcat = data[[subcat_name]])
#>     if (!is.factor(data_temp$subcat)) {
#>       data_temp$subcat = factor(data_temp$subcat)
#>     }
#>     if (!(subcat_level %in% levels(data_temp$subcat))) {
#>       stop("subcat_level does not match levels of subcat_name")
#>     }
#>     data_temp = subset(data_temp, data_temp$subcat == subcat_level)
#>   }
#>   if (!is.numeric(data_temp$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   if (!is.character(emissions)) {
#>     emissions = colnames(data[emissions])
#>   }
#>   if (!is.character(sources)) {
#>     sources = colnames(data[sources])
#>   }

```

```

#>   }
#>   analysis_env = list2env(list(HAP = HAP, emissions = emissions,
#>     subcat_name = subcat_name, subcat_level = subcat_level,
#>     data = data_temp, CAA_section = CAA_section, national_N = national_N,
#>     meas_unit = meas_unit, sources = sources, type = type,
#>     significance = significance, future_runs = future_runs,
#>     more_args = ...), parent = as.environment(2))
#>   return(analysis_env)
#> }
#> <bytecode: 0x0000028301d8fdb8>
#> <environment: namespace:UPLforOAR>

```

JAGS Model Scripts

```

#> [1] "# Beta"
#> [2] "    model {"
#> [3] "    # priors"
#> [4] "        # note that both can be pos or neg, but the minimum is always -1"
#> [5] "        beta_em ~ dunif(-1, (0.25 - 1 + 0.25 / sdY^2 + 0.25^3 / sdY^2 - 2 *"
#> [6] "            0.25^2 / sdY^2) * 1.25)"
#> [7] "        alpha_em ~ dunif(-1, (0.75^2 / sdY^2 - 0.75^3 / sdY^2 - 0.75) * 1.25)"
#> [8] ""
#> [9] "    #likelihood"
#> [10] "        for (i in 1:length(emission_xi)) {"
#> [11] "            emission_xi[i] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [12] "            pdf_obs[i] = dbeta(emission_xi[i], alpha_em, beta_em)"
#> [13] "        }"
#> [14] ""
#> [15] "    # derived quantities"
#> [16] "    emission_mean = alpha_em / (alpha_em + beta_em)"
#> [17] "    # predict new emission tests"
#> [18] "        for (k in 1:n_draws){"
#> [19] "            emission_hat[k] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [20] "        }"
#> [21] "        for (h in 1:n_x_hat){"
#> [22] "            pdf_hat[h] = dbeta(x_hat[h], alpha_em, beta_em)"
#> [23] "        }"
#> [24] "    }"
#> [1] "# Gamma"
#> [2] "    model {"
#> [3] "    # priors"
#> [4] "        rate_em ~ dunif(0, maxY / (sdY^2)) #must be positive"
#> [5] "        shape_em ~ dunif(0, (maxY^2) / (sdY^2)) #must be positive"
#> [6] ""
#> [7] "    #likelihood"
#> [8] "        for (i in 1:length(emission_xi)) {"
#> [9] "            emission_xi[i] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [10] "            pdf_obs[i] = dgamma(emission_xi[i], shape_em, rate_em)"
#> [11] "        }"
#> [12] ""
#> [13] "    # derived quantities"
#> [14] "    # predict new emission tests"
#> [15] "        for (k in 1:n_draws){"
#> [16] "            emission_hat[k] ~ dgamma(shape_em, rate_em)T(minY, maxY)"

```

```

#> [17] "          }"
#> [18] "          for (h in 1:n_x_hat){"
#> [19] "              pdf_hat[h] = dgamma(x_hat[h], shape_em, rate_em)"
#> [20] "          }"
#> [21] "      }"
#> [1] "# Lognormal"
#> [2] "      model {"
#> [3] "          # priors"
#> [4] "              sd_ln ~ dunif( 0.001 * sdOfLogY, 1000 * sdOfLogY )"
#> [5] "              tau_ln = 1 / (sd_ln^2)"
#> [6] ""
#> [7] "              #this is in log-space, so it can be negative"
#> [8] "              u_ln ~ dnorm( meanOfLogY, 0.001 / sdOfLogY^2 )"
#> [9] ""
#> [10] "      #likelihood"
#> [11] "          for (i in 1:length(emission_xi)){"
#> [12] "              emission_xi[i] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [13] "              pdf_obs[i] = dlnorm(emission_xi[i], u_ln, tau_ln)"
#> [14] "          }"
#> [15] ""
#> [16] "      # derived quantities"
#> [17] "      # predict new emission tests"
#> [18] "          for (k in 1:n_draws){"
#> [19] "              emission_hat[k] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [20] "          }"
#> [21] "          for (h in 1:n_x_hat){"
#> [22] "              pdf_hat[h] = dlnorm(x_hat[h], u_ln, tau_ln)"
#> [23] "          }"
#> [24] "      }"
#> [1] "# Normal"
#> [2] "      model {"
#> [3] "          # priors"
#> [4] "              emission_sd ~ dunif(0.001 * sdY, 1000 * sdY)"
#> [5] "              emission_mean ~ dunif(0,maxY)"
#> [6] "              tau_em = 1 / (emission_sd^2)"
#> [7] ""
#> [8] "      #likelihood"
#> [9] "          for (i in 1:length(emission_xi)) {"
#> [10] "              emission_xi[i] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [11] "              pdf_obs[i] = dnorm(emission_xi[i], emission_mean, tau_em)"
#> [12] "          }"
#> [13] ""
#> [14] "      # derived quantities"
#> [15] "      # predict new emission tests"
#> [16] "          for (k in 1:n_draws){"
#> [17] "              emission_hat[k] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [18] "          }"
#> [19] "          for (h in 1:n_x_hat){"
#> [20] "              pdf_hat[h] = dnorm(x_hat[h], emission_mean, tau_em)"
#> [21] "          }"
#> [22] "      }"
#> [1] "# Skewed"
#> [2] "      data {"
#> [3] "          for(i in 1:length(emission_xi)){"

```

```

#> [4] "          zeros[i] = 0"
#> [5] "      }"
#> [6] "  }"
#> [7] "  model {"
#> [8] "    # priors"
#> [9] "      omega ~ dunif(0, 100 * maxY) #must be positive"
#> [10] "      xi ~ dnorm(0, 1 / (100 * maxY))"
#> [11] "      alpha ~ dunif(-100, 100) # this is a very wide range"
#> [12] ""
#> [13] "    #likelihood"
#> [14] "      for (i in 1:length(emission_xi)) {"
#> [15] "        L[i] = ((2 / omega)"
#> [16] "          * dnorm((emission_xi[i] - xi) / omega, 0, 1)"
#> [17] "          * pnorm(alpha * (emission_xi[i] - xi) / omega, 0, 1))"
#> [18] "        zeros[i] ~ dpois(-log(L[i]) + 10000)"
#> [19] "      }"
#> [20] "    }"
#> [1] "# Beta manual priors"
#> [2] "  model {"
#> [3] "    # priors"
#> [4] "      # note that both can be pos or neg, but the minimum is always -1"
#> [5] "      alpha_em ~ dunif(low1, up1)"
#> [6] "      beta_em ~ dunif(low2, up2)"
#> [7] ""
#> [8] "    #likelihood"
#> [9] "      for (i in 1:length(emission_xi)) {"
#> [10] "        emission_xi[i] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [11] "        pdf_obs[i] = dbeta(emission_xi[i], alpha_em, beta_em)"
#> [12] "      }"
#> [13] ""
#> [14] "    # derived quantities"
#> [15] "    emission_mean = alpha_em / (alpha_em + beta_em)"
#> [16] "    # predict new emission tests"
#> [17] "    for (k in 1:n_draws){"
#> [18] "      emission_hat[k] ~ dbeta(alpha_em, beta_em)T(minY, maxY)"
#> [19] "    }"
#> [20] "    for (h in 1:n_x_hat){"
#> [21] "      pdf_hat[h] = dbeta(x_hat[h], alpha_em, beta_em)"
#> [22] "    }"
#> [23] "  }"
#> [1] "# Gamma manual priors"
#> [2] "  model {"
#> [3] "    # priors"
#> [4] "      rate_em ~ dunif(low1, up1) #must be positive"
#> [5] "      shape_em ~ dunif(low2, up2) #must be positive"
#> [6] ""
#> [7] "    #likelihood"
#> [8] "      for (i in 1:length(emission_xi)) {"
#> [9] "        emission_xi[i] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [10] "        pdf_obs[i] = dgamma(emission_xi[i], shape_em, rate_em)"
#> [11] "      }"
#> [12] ""
#> [13] "    # derived quantities"
#> [14] "    # predict new emission tests"

```

```

#> [15] "          for (k in 1:n_draws){"
#> [16] "              emission_hat[k] ~ dgamma(shape_em, rate_em)T(minY, maxY)"
#> [17] "          }"
#> [18] "          for (h in 1:n_x_hat){"
#> [19] "              pdf_hat[h] = dgamma(x_hat[h], shape_em, rate_em)"
#> [20] "          }"
#> [21] "      }"
#> [1] "# Lognormal manual priors"
#> [2] "      model {"
#> [3] "          # priors"
#> [4] "              sd_ln ~ dunif(low1, up1)"
#> [5] "              u_ln ~ dnorm(low2, up2) #this is in log-space, so it can be negative"
#> [6] "              tau_ln = 1 / (sd_ln^2)"
#> [7] ""
#> [8] "          #likelihood"
#> [9] "              for (i in 1:length(emission_xi)) {"
#> [10] "                  emission_xi[i] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [11] "                  pdf_obs[i] = dlnorm(emission_xi[i], u_ln, tau_ln)"
#> [12] "              }"
#> [13] ""
#> [14] "          # derived quantities"
#> [15] "          # predict new emission tests"
#> [16] "              for (k in 1:n_draws){"
#> [17] "                  emission_hat[k] ~ dlnorm(u_ln, tau_ln)T(minY, maxY)"
#> [18] "              }"
#> [19] "              for (h in 1:n_x_hat){"
#> [20] "                  pdf_hat[h] = dlnorm(x_hat[h], u_ln, tau_ln)"
#> [21] "              }"
#> [22] "          }"
#> [1] "# Normal manual priors"
#> [2] "      model {"
#> [3] "          # priors"
#> [4] "              emission_sd ~ dunif(low1, up1)"
#> [5] "              emission_mean ~ dunif(low2, up2)"
#> [6] "              tau_em = 1 / (emission_sd^2)"
#> [7] ""
#> [8] "          #likelihood"
#> [9] "              for (i in 1:length(emission_xi)) {"
#> [10] "                  emission_xi[i] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [11] "                  pdf_obs[i] = dnorm(emission_xi[i], emission_mean, tau_em)"
#> [12] "              }"
#> [13] ""
#> [14] "          # derived quantities"
#> [15] "          # predict new emission tests"
#> [16] "              for (k in 1:n_draws){"
#> [17] "                  emission_hat[k] ~ dnorm(emission_mean, tau_em)T(minY, maxY)"
#> [18] "              }"
#> [19] "              for (h in 1:n_x_hat){"
#> [20] "                  pdf_hat[h] = dnorm(x_hat[h], emission_mean, tau_em)"
#> [21] "              }"
#> [22] "          }"
#> [1] "# Skewed manual priors"
#> [2] "      data {"
#> [3] "          for(i in 1:length(emission_xi)) {"

```

```

#> [4] "          zeros[i] = 0"
#> [5] "      }"
#> [6] "  }"
#> [7] "  model {"
#> [8] "    # priors"
#> [9] "      omega ~ dunif(low1, up1) #must be positive"
#> [10] "      xi ~ dunif(low2, up2)"
#> [11] "      alpha ~ dunif(low3, up3) # this is a very wide range"
#> [12] ""
#> [13] "    #likelihood"
#> [14] "      for (i in 1:length(emission_xi)) {"
#> [15] "        L[i] = ((2 / omega)"
#> [16] "          * dnorm((emission_xi[i] - xi) / omega, 0, 1)"
#> [17] "          * pnorm(alpha * (emission_xi[i] - xi) / omega, 0, 1))"
#> [18] "        zeros[i] ~ dpois(-log(L[i]) + 10000)"
#> [19] "      }"
#> [20] "    }"

```


Appendix C: System and package versions

```
#> R version 4.4.0 (2024-04-24 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 22631)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] grid      stats      graphics  grDevices utils      datasets  methods
#> [8] base
#>
#> other attached packages:
#> [1] knitr_1.50      UPLforOAR_0.1.0 sfsmisc_1.1-22  np_0.60-18
#> [5] EnvStats_3.1.0 lubridate_1.9.4 forcats_1.0.1   stringr_1.5.2
#> [9] purrr_1.1.0     readr_2.1.5      tidyr_1.3.1     tibble_3.3.0
#> [13] tidyverse_2.0.0 ggplot2_4.0.0    dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6      xfun_0.53          lattice_0.22-6
#> [4] tzdb_0.5.0        numDeriv_2016.8-1.1 quadprog_1.5-8
#> [7] vctr_0.6.5        tools_4.4.0        generics_0.1.4
#> [10] stats4_4.4.0      parallel_4.4.0     pkgconfig_2.0.3
#> [13] Matrix_1.7-0      RColorBrewer_1.1-3 S7_0.2.0
#> [16] lifecycle_1.0.4   cubature_2.1.4     compiler_4.4.0
#> [19] farver_2.1.2      MatrixModels_0.5-4 mnormt_2.1.1
#> [22] runjags_2.2.2-5   SparseM_1.84-2     quantreg_6.1
#> [25] htmltools_0.5.8.1 yaml_2.3.10        pillar_1.11.1
#> [28] crayon_1.5.3      MASS_7.3-60.2      boot_1.3-30
#> [31] tidyselect_1.2.1  digest_0.6.37      stringi_1.8.7
#> [34] labeling_0.4.3    splines_4.4.0      fastmap_1.2.0
#> [37] cli_3.6.5         magrittr_2.0.4     patchwork_1.3.2
#> [40] survival_3.5-8    sn_2.1.1           withr_3.0.2
#> [43] scales_1.4.0      bit64_4.6.0-1      timechange_0.3.0
#> [46] rmarkdown_2.30    matrixStats_1.5.0  bit_4.6.0
#> [49] rjags_4-17        hms_1.1.3          coda_0.19-4.1
#> [52] evaluate_1.0.5    rlang_1.1.6        Rcpp_1.1.0
#> [55] glue_1.8.0        rstudioapi_0.17.1  vroom_1.6.6
#> [58] R6_2.6.1
```

Appendix D: Rmd Code Chunks

Main document code chunks:

```
# Below is an example of loading data, sorting and selecting to top performers,
# and calculating the correct distribution and UPL for existing source and NSPS.
# If there are multiple HAP or subcategories, simply repeat with code block for
# those data, making sure to use unique names so you don't overwrite the results.

if (params$CAA_section == 112){
  CAA_text = 'CAA Section 112(d)(3)'
} else if (params$CAA_section == 129){
  CAA_text = 'CAA Section 129(a)(2)'
}

# If using a single data set then upload with params:
dat_industry = read_csv(params$data)

# Or load manually:
dat_industry = read_csv('Example_data4.csv')
# If it is not in your working directory, you will need to include the file path:
# dat_industry = read_csv('USER/name/data/Example_data4.csv')

# In this example data, column 1 is the sources, there are two HAPS, Chemical X
# and Chemical Y in columns 2 and 3 in your data, named 'chemX' and 'chemY', and
# one subcategory, Size, with levels A and B.

# Let's suppose we want Existing source UPL calculations for both HAP's, the Size
# subcategory only applies to one HAP (Chemical X), and we also want New source on
# subcategory B. This is four analyses: existing source UPL's for Chemical X
# subcategory A & B, and Chemical Y, and new source UPL for Chemical X subcategory B.

# Next, designate lists (of length four, the number of calculations) for the
# emissions, sources, and subcategory for which
# you will be doing each UPL calculation, and whether or not it is for an existing
# or new source, or as-is (no subsetting):
subcat_list = c('Size', 'Size', 'Size', NA)
subcat_levels = c('A', 'B', 'B', NA)
type_list = c('Existing', 'Existing', 'New', 'Existing')
HAP_list = c('Chemical X', 'Chemical X', 'Chemical X', 'Chemical Y')
emiss_list = c('chemX', 'chemX', 'chemX', 'chemY')
meas_unit_list = c('ppm', 'ppm', 'ppm', 'mg/m3/s')
sources = 1

# Note that HAP_list are the names as we want them in text or labels
# the lists for emissions, sources, and subcat_list can be the variable names
# as a string or indicate the column number in the data set.
# For example, let's say your sources are in column 1 and are named 'boiler units'
# R doesn't like spaces and will replace them (and special characters) with periods.
# We can designate the sources by name ('boiler.units') or position (1).

# We will do the default 3 future runs and 0.99 significance for all four UPLs:
future_run_list = c(3, 3, 3, 3)
sig_list = c(0.99, 0.99, 0.99, 0.99)
```

```

# For the Bayes methods, we will assume you have already done an exploratory
# analysis to determine the best distribution for each UPL calculation, and only
# need to include the one we are using in this report.
distr_list = c('Skewed', 'Skewed', 'Skewed', 'Normal')

# Additional optional arguments to Bayesian_UPL() can be passed as well.
# For example, suppose you want to a gaussian kernel for density figures but only for
# the last UPL calculation for Chemical Y: kernel_list = c(NA, NA, NA, 'gaussian1')
# Then include below: kernel_arg = kernel_list[i]

# Now we can create a list of environments to populate the .Rmd document:
env_list = list()
for (i in 1:length(HAP_list)){
  env_temp = make_environment(data = dat_industry,
                             distr = distr_list[i],
                             HAP = HAP_list[i],
                             emissions = emiss_list[i],
                             sources = sources,
                             subcat_name = subcat_list[i],
                             subcat_level = subcat_levels[i],
                             type = type_list[i],
                             meas_unit = meas_unit_list[i],
                             CAA_section = params$CAA_section,
                             national_N = params$national_N,
                             future_runs = future_run_list[i],
                             significance = sig_list[i])
  env_list = c(env_list, env_temp)
}

# You also use the make_environment() function outside of the for loop
col1=c('Subcategory 1','Subcategory 2','Subcategory 3',
       'Subcategory 4','Subcategory 5')
col2=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col3=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col4=c('HAP 1, HAP 2, X, Y, Z', 'HAP 1, X, Y, Z', 'HAP 1, HAP 2, HAP 3,
       X, Y, Z', 'None', 'X, Y, Z')
table1=tibble(col1=col1,col2=col2,col3=col3,col4=col4)
table1%>%knitr::kable(
  caption="Here is an example of a table in case you need one",
  col.names=c(' ',
              'Existing MACT floor (old units)',
              'Existing MACT floor (new units)',
              'New MACT floor'))

# check if any UPL results from new and existing source pairs need to be set
# such that the new UPL <= existing UPL.
order_combn = combn(seq(1,length(HAP_list), by = 1), 2)
HAP_combn = combn(HAP_list, 2)
subcat_combn = combn(subcat_list, 2)
level_combn = combn(subcat_levels, 2)
meas_combn = combn(meas_unit_list, 2)

check_HAPs = HAP_combn[1, ] == HAP_combn[2, ]
check_subcat = subcat_combn[1, ] == subcat_combn[2, ]
check_levels = level_combn[1, ] == level_combn[2, ]
check_units = meas_combn[1, ] == meas_combn[2, ]

```

```

for (i in 1:length(check_HAPs)){
  if (all(check_HAPs[i], check_subcat[i], check_levels[i], check_units[i])){
    pair1 = order_combn[1,i]
    pair2 = order_combn[2,i]
    type1 = env_list[[pair1]]$type
    type2 = env_list[[pair2]]$type
    UPL1 = env_list[[pair1]]$results$fit_table$UPL
    UPL2 = env_list[[pair2]]$results$fit_table$UPL
    if ((type1 == "New") & (UPL1 > UPL2) & (type2 == 'Existing')){
      env_list[[pair1]]$UPL_old = UPL1
      env_list[[pair1]]$results$fit_table$UPL = UPL2
      env_list[[pair1]]$UPL_warn = TRUE
    } else if ((type2 == "New") & (UPL2 > UPL1) & (type1 == 'Existing')){
      env_list[[pair2]]$UPL_old = UPL2
      env_list[[pair2]]$results$fit_table$UPL = UPL1
      env_list[[pair2]]$UPL_warn = TRUE
    }
  }
}
UPL_list = c()
for (i in 1:length(env_list)){
  UPL_temp = signif(env_list[[i]]$results$fit_table$UPL, digits = 4)
  UPL_list = c(UPL_list, UPL_temp)
}
dat_UPL = tibble(HAP = HAP_list,
                  Subcategory = subcat_list,
                  Level = subcat_levels,
                  Type = type_list,
                  UPL = UPL_list,
                  Sig = sig_list,
                  Runs = future_run_list)
dat_UPL %>% knitr::kable(
  caption = "Upper Predictive Limit (UPL) results for MACT floors in
(measurement units)", escape = FALSE,
  col.names = c("HAP", "Subcategory<br/>Name", "Subcategory<br/>Level", "Source Type",
                "UPL", "Significance", "No. Future<br/>Runs"))
part1 = setup_likelihoood(distribution = distr, data = df,
                          emissions = 'emissions', RNG.state = RNG.state,
                          manual_prior = manual_prior, random = FALSE,
                          prior_list = prior_list)
part2 = run_likelihoood(model_input = part1, future_runs = future_runs,
                        xvals = xvals, maxY = maxY, minY = minY)
part3 = converge_figs(distribution = distr, jags_model_run = part2)
part3
part1 = setup_likelihoood(distribution = distr, data = df,
                          emissions = 'emissions', RNG.state = RNG.state,
                          manual_prior = manual_prior, random = FALSE,
                          prior_list = prior_list)
part2 = run_likelihoood(model_input = part1, future_runs = future_runs,
                        xvals = xvals, maxY = maxY, minY = minY)
part3 = converge_figs(distribution = distr, jags_model_run = part2)
part3
part1 = setup_likelihoood(distribution = distr, data = df,
                          emissions = 'emissions', RNG.state = RNG.state,

```

```

        manual_prior = manual_prior, random = FALSE,
        prior_list = prior_list)
part2 = run_likelihood(model_input = part1, future_runs = future_runs,
                      xvals = xvals, maxY = maxY, minY = minY)
part3 = converge_figs(distribution = distr, jags_model_run = part2)
part3
part1 = setup_likelihood(distribution = distr, data = df,
                        emissions = 'emissions', RNG.state = RNG.state,
                        manual_prior = manual_prior, random = FALSE,
                        prior_list = prior_list)
part2 = run_likelihood(model_input = part1, future_runs = future_runs,
                      xvals = xvals, maxY = maxY, minY = minY)
part3 = converge_figs(distribution = distr, jags_model_run = part2)
part3

```

Child document code chunks:

Analysis_template.Rmd

```

``{r, echo = FALSE, appendix = FALSE}
if (type == 'Existing'){
  df = data
  if (any(df$emissions == 0)){
    zero_warn = TRUE
  } else {
    zero_warn = FALSE
  }
  df = MACT_existing(CAA_section = CAA_section, data = df,
                    emissions = 'emissions', sources = 'sources',
                    national_N = national_N)
  df_mean = df %>% group_by(sources) %>%
    summarize(means = mean(emissions),
              counts = n())

  if (all(df_mean$counts == 1)){
    source_fig = FALSE
  } else {
    source_fig = TRUE
  }
} else if (type == "New"){
  df = data
  if (any(df$emissions == 0)){
    zero_warn = TRUE
  } else {
    zero_warn = FALSE
  }
  source_fig = FALSE
  df = MACT_new(data = df, emissions = 'emissions', sources = 'sources')
} else if (type == "As-is"){
  df = data
  if (any(df$emissions == 0)){
    zero_warn = TRUE
  } else {
    zero_warn = FALSE
  }
  df_mean = df %>% group_by(sources) %>%

```

```

        summarize(means = mean(emissions),
                  counts = n())

    if (all(df_mean$counts == 1)){
      source_fig = FALSE
    } else {
      source_fig = TRUE
    }
  }
  UPL_warn = FALSE
  best_source = as.character(df$sources)[1]
  df$ln_emiss = log(df$emissions)
  df$ln_emiss = replace(df$ln_emiss, !is.finite(df$ln_emiss), NA)
  if (!is.na(subcat_name)){
    subcat_text = paste0('for ', subcat_name, ' subcategory ', subcat_level)
  } else {
    subcat_text = ''
  }
  if (!exists('maxY')){
    maxY = 3 * max(df$emissions)
  } else if (is.na(maxY)){
    maxY = 3 * max(df$emissions)
  }
  if (!exists('minY')){
    minY = 0
  } else if (is.na(minY)){
    minY = 0
  }
  if (!exists('xvals')){
    xvals = NULL
  } else if (!is.null(xvals)){
    if (is.na(xvals)){
      xvals = NULL
    }
  }
  if (!exists('up')){
    up = Inf
  } else if (is.na(up)){
    up = Inf
  }
  if (!exists('low')){
    low = 0
  } else if (is.na(low)){
    low = 0
  }
  if (!exists('kernel_arg')){
    kernel_arg = 'gamma'
  } else if (is.null(kernel_arg)){
    kernel_arg = 'gamma'
  } else if (is.na(kernel_arg)){
    kernel_arg = 'gamma'
  }
  if (!exists('bw')){
    bw = NULL
  } else if (!is.null(bw)){

```

```

    if (is.na(bw)){
      bw = NULL
    }
  }
  if (!exists('manual_prior')){
    manual_prior = FALSE
  } else if (is.na(manual_prior)){
    manual_prior = FALSE
  }
  if (!exists('prior_list')){
    prior_list = NULL
  } else if (!is.null(prior_list)){
    if (is.na(prior_list)){
      prior_list = NULL
    }
  }
}
if (!exists('RNG.state')){
  RNG.state = NULL
} else if (!is.null(RNG.state)){
  if (is.na(RNG.state)){
    RNG.state = NULL
  }
}
}
results = Bayesian_UPL(data = df, distr_list = distr,
                      emissions = 'emissions',
                      future_runs = future_runs,
                      significance = significance, RNG.state = RNG.state,
                      maxY = maxY, minY = minY, xvals = xvals,
                      up = up, low = low, kernel = kernel_arg, bw = bw,
                      convergence_report = FALSE, random = FALSE,
                      manual_prior = manual_prior, prior_list = prior_list)
...

```

Bayes_MCMC_settings.Rmd

```
### `r type` Source `r HAP` `r subcat_text`
```

The selected distribution for the ``r type`` source data set for ``r HAP`` ``r subcat_text``

- was the ``r distr`` distribution, with the likelihood truncated such that it was zero
- outside the bounds of (``r minY``, ``r signif(maxY, 3)``) emissions. The parameter prior
- distributions, MCMC initial chain values, and Gelman-Rubin diagnostics for
- convergence are listed in the table below, followed by the random number generator
- (RNG) state that was used.

```

```${r, echo = FALSE, appendix = FALSE}
This R chunk uses the initial value settings copied from setup_likelihood().
If setup_likelihood() has these calculations changed, it will need to be updated here.
Eventually, it would be ideal to have a separate function to set initial values
based on data and distribution, then setup_likelihood() would call it and it
would be called here, so updates only happen in a single place.
if (!manual_prior){
 if (distr == 'Gamma'){
 sdY = sd(df$emissions)
 shape = mean(df$emissions)^2 / sdY^2
 }
}

```

```

rate = mean(df$emissions) / sdY^2
parameter = c('Rate', 'Shape')
prior = c(paste0('~dunif (0, ', signif(maxY / (sdY^2), 3), ')'),
 paste0('~dunif (0, ', signif((maxY^2) / (sdY^2), 3), ')))
chain1 = c(signif(rate, 3), signif(shape, 3))
chain2 = c(signif(1.5 * rate, 3), signif(0.5 * shape, 3))
chain3 = c(signif(0.5 * rate, 3), signif(1.5 * shape, 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
} else if (distr == 'Normal'){
sdY = sd(df$emissions)
meanY = mean(df$emissions)
parameter = c('Mean', 'Standard deviation')
prior = c(paste0('~dunif (0, ', signif(maxY, 3), ')'),
 paste0('~dunif (', signif(0.001 * sdY, 3), ' ',
 signif(1000 * sdY, 3), ')))
chain1 = c(signif(meanY, 3), signif(sdY, 3))
chain2 = c(signif(1.5 * meanY, 3), signif(0.5 * sdY, 3))
chain3 = c(signif(0.5 * meanY, 3), signif(1.5 * sdY, 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
} else if (distr == 'Skewed'){
sdY = sd(df$emissions)
meanY = mean(df$emissions)
skew1 = min(0.99, abs((1 / length(df$emissions)) *
 sum(((df$emissions - meanY) / sdY)^3)))
delta = sqrt((pi / 2) * ((abs(skew1)^(2 / 3)) / ((abs(skew1)^(2 / 3)) +
 ((4 - pi) / 2)^(2 / 3))))

delta = delta * abs(skew1) / skew1
alpha = delta / sqrt(1 - delta^2)
omega = sdY / sqrt(1 - 2 * delta^2 / pi)
xi = meanY - omega * delta * sqrt(2 / pi)
parameter = c('Omega', 'Xi', 'Alpha')
prior = c(paste0('~dunif (0, ', signif(maxY, 3), ')'),
 paste0('~dnorm (0, ', signif(1 / (100 * maxY), 3), ')'),
 '~dunif (-100, 100)')
chain1 = c(signif(xi, 3), signif(omega, 3), signif(alpha, 3))
chain2 = c(signif(1.5 * xi, 3), signif(0.5 * omega, 3), signif(0.5 * alpha, 3))
chain3 = c(signif(0.5 * xi, 3), signif(1.5 * omega, 3), signif(1.5 * alpha, 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3),
 signif(results$conv_output$gelman_diag[3], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2],
 results$conv_output$convYN[3])
} else if (distr == "Lognormal"){
sdY = sd(df$ln_emiss)
meanY = mean(df$ln_emiss)
parameter = c('Mean', 'Standard deviation')
prior = c(paste0('~dnorm (', meanY, ' ',
 signif(0.001 / (sdY^2), 3), ')'),

```



```

 paste0('~dunif (', signif(0.001 * sdY, 3), ' ',
 signif(1000 * sdY, 3), ')'))
chain1 = c(signif(meanY, 3), signif(sdY, 3))
chain2 = c(signif(1.5 * meanY, 3), signif(0.5 * sdY, 3))
chain3 = c(signif(0.5 * meanY, 3), signif(1.5 * sdY, 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
} else if (distr == 'Beta'){
 sdY = sd(df$emissions)
 meanY = mean(df$emissions)
 alpha = meanY^2 / sdY^2 - meanY^3 / sdY^2 - meanY
 beta = meanY - 1 + meanY / sdY^2 + meanY^3 / sdY^2 - 2 * meanY^2 / sdY^2
 parameter = c('Lambda', 'Tau')
 prior = c(paste0('~dunif (-1, ', signif((0.75^2 / sdY^2 - 0.75^3 / sdY^2 - 0.75)
 * 1.25, 3), ')'),
 paste0('~dunif (-1, ', signif((0.25 - 1 + 0.25 / sdY^2 +
 0.25^3 / sdY^2 - 2 * 0.25^2 / sdY^2)
 * 1.25, 3), ')'))

 chain1 = c(signif(alpha, 3), signif(beta, 3))
 chain2 = c(signif(1.5 * alpha, 3), signif(0.5 * beta, 3))
 chain3 = c(signif(0.5 * alpha, 3), signif(1.5 * beta, 3))
 diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
 convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
}
} else if (manual_prior){
 if (distr == "Normal"){
 parameter = c('Mean', 'Standard deviation')
 prior = c(paste0('~dunif (', prior_list[3], ' ', prior_list[4], ')'),
 paste0('~dunif (', prior_list[1], ' ', prior_list[2], ')'))
 chain1 = c(signif(mean(c(prior_list[3], prior_list[4])), 3),
 signif(mean(c(prior_list[1], prior_list[2])), 3))
 chain2 = c(signif(0.9 * prior_list[4], 3), signif(0.9 * prior_list[2], 3))
 chain3 = c(signif(1.1 * prior_list[3], 3), signif(1.1 * prior_list[1], 3))
 diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
 convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
 } else if (distr == 'Lognormal'){
 if (prior_list[4] < 0){
 initial1 = 1.1 * prior_list[4]
 } else if (prior_list[4] > 0){
 initial1 = 0.9 * prior_list[4]
 }
 if (prior_list[3] < 0){
 initial2 = 0.9 * prior_list[3]
 } else if (prior_list[3] > 0){
 initial2 = 1.1 * prior_list[3]
 }
 parameter = c('Mean', 'Standard deviation')
 prior = c(paste0('~dunif (', prior_list[3], ' ', prior_list[4], ')'),

```

```

 paste0('~dunif (', prior_list[1], ', ', prior_list[2], ')'))
chain1 = c(signif(mean(c(prior_list[3], prior_list[4])), 3),
 signif(mean(c(prior_list[1], prior_list[2])), 3))
chain2 = c(signif(initial1, 3), signif(0.9 * prior_list[2], 3))
chain3 = c(signif(initial2, 3), signif(1.1 * prior_list[1], 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
} else if (distr == 'Skewed'){
 if (prior_list[4] < 0){
 initial1 = 1.1 * prior_list[4]
 } else if (prior_list[4] > 0){
 initial1 = 0.9 * prior_list[4]
 }
 if (prior_list[3] < 0){
 initial2 = 0.9 * prior_list[3]
 } else if (prior_list[3] > 0){
 initial2 = 1.1 * prior_list[3]
 }
 if (prior_list[6] < 0){
 initial3 = 1.1 * prior_list[6]
 } else if (prior_list[6] > 0){
 initial3 = 0.9 * prior_list[6]
 }
 if (prior_list[5] < 0){
 initial4 = 0.9 * prior_list[5]
 } else if (prior_list[5] > 0){
 initial4 = 1.1 * prior_list[5]
 }
 parameter = c('Omega', 'Xi', 'Alpha')
 prior = c(paste0('~dunif (', prior_list[1], ', ', prior_list[2], ')'),
 paste0('~dunif (', prior_list[3], ', ', prior_list[4], ')'),
 paste0('~dunif (', prior_list[5], ', ', prior_list[6], ')'))
 chain1 = c(signif(mean(c(prior_list[1], prior_list[2])), 3),
 signif(mean(c(prior_list[3], prior_list[4])), 3),
 signif(mean(c(prior_list[5], prior_list[6])), 3))
 chain2 = c(signif(1.1 * prior_list[1], 3),
 signif(initial1, 3),
 signif(initial4, 3))
 chain3 = c(signif(0.9 * prior_list[2], 3),
 signif(initial2, 3),
 signif(initial3, 3))
 diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3),
 signif(results$conv_output$gelman_diag[3], 3))
 convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2],
 results$conv_output$convYN[3])
} else if (distr == 'Gamma'){
 parameter = c('Rate', 'Shape')
 prior = c(paste0('~dunif (', prior_list[1], ', ', prior_list[2], ')'),
 paste0('~dunif (', prior_list[3], ', ', prior_list[4], ')'))
 chain1 = c(signif(mean(c(prior_list[1], prior_list[2])), 3),

```

```

 signif(mean(c(prior_list[3], prior_list[4])), 3))
chain2 = c(signif(1.1 * prior_list[1], 3), signif(1.1 * prior_list[3], 3))
chain3 = c(signif(0.9 * prior_list[2], 3), signif(0.9 * prior_list[4], 3))
diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
} else if (distr == 'Beta'){
 if (prior_list[2] < 0){
 initial1 = 1.1 * prior_list[2]
 } else if (prior_list[2] > 0){
 initial1 = 0.9 * prior_list[2]
 if (initial1 < (-1)){
 initial1 = -0.99999
 }
 }
 if (prior_list[1] < 0){
 initial2 = 0.9 * prior_list[1]
 } else if (prior_list[1] > 0){
 initial2 = 1.1 * prior_list[1]
 if (initial1 < (-1)){
 initial1 = -0.99999
 }
 }
 if (prior_list[4] < 0){
 initial3 = 1.1 * prior_list[4]
 } else if (prior_list[4] > 0){
 initial3 = 0.9 * prior_list[4]
 if (initial1 < (-1)){
 initial1 = -0.99999
 }
 }
 if (prior_list[3] < 0){
 initial4 = 0.9 * prior_list[3]
 } else if (prior_list[3] > 0){
 initial4 = 1.1 * prior_list[3]
 if (initial1 < (-1)){
 initial1 = -0.99999
 }
 }
 parameter = c('Lambda', 'Tau')
 prior = c(paste0('~dunif (', prior_list[1], ', ', prior_list[2], ')'),
 paste0('~dunif (', prior_list[3], ', ', prior_list[4], ')'))
 chain1 = c(signif(mean(c(prior_list[1], prior_list[2])), 3),
 signif(mean(c(prior_list[3], prior_list[4])), 3))
 chain2 = c(signif(initial1, 3), signif(initial3, 3))
 chain3 = c(signif(initial2, 3), signif(initial4, 3))
 diagnostics = c(signif(results$conv_output$gelman_diag[1], 3),
 signif(results$conv_output$gelman_diag[2], 3))
 convYN = c(results$conv_output$convYN[1],
 results$conv_output$convYN[2])
}
}
param_tab = tibble(parameter, prior, chain1, chain2, chain3,

```

```

 diagnostics, convYN)
param_tab %>% knitr::kable(
 caption = paste0("Parameter information for the ", distr, " distribution
 UPL calculation for ", type, " source", HAP, " ", subcat_text),
 col.names = c('Parameter',
 'Prior',
 'Chain 1 initial value',
 'Chain 2 initial value',
 'Chain 3 initial value',
 'Gelman-Rubin diagnostic',
 'Converged?'))
...

```{r, appendix = FALSE, echo = FALSE}
state_out = results$state
chains = c('Chain 1', 'Chain 2', 'Chain 3')
rng_name = c(state_out[[1]]$RNG.name,
             state_out[[2]]$RNG.name,
             state_out[[3]]$RNG.name)
if (is.null(RNG.state)){
  rng_state = c('seed = 5', 'seed = 12', 'seed = 151')
} else {
  rng_state = c(paste(state_out[[1]]$RNG.state, collapse=', '),
               paste(state_out[[2]]$RNG.state, collapse=', '),
               paste(state_out[[3]]$RNG.state, collapse=', '))
}
RNG1 = tibble(chains, rng_name, rng_state)
RNG1 %>% knitr::kable(
  caption="Random number generator (RNG) names and states used in MCMC chains",
  col.names=c('MCMC Chain',
              'Name',
              'State'))
...

```

Source_variance.Rmd

```

### `r type` Source `r HAP` `r subcat_text`

The final data set for the `r HAP` `r type` source MACT floor `r subcat_text` included
→ `r nrow(df)` test runs from `r length(unique(df$sources))` sources. The best
→ performer for the new source calculation was `r best_source` with an average emission
→ of `r signif(mean(df$emissions), digits = 4)` (`r meas_unit`) from `r nrow(df)` test
→ runs.

```{r, echo = FALSE, fig.align = "center", fig.height = 4, appendix = FALSE, eval =
→ source_fig, fig.cap = "Observation density of emissions by source type. Source
→ averages are indicated by vertical lines."}
if (!is.na(subcat_name)){
 ggplot(data = df, aes(emissions, fill = sources))+
 geom_density(alpha = 0.5, bounds = c(0, Inf))+
 ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
 ggtitle(paste0(subcat_name, " ", subcat_level, " ", type,
 " source test runs"))+
 multi_source_theme()+labs(fill = 'Top sources',

```

```

 color = 'Top sources')+
geom_vline(aes(xintercept = means, color = sources),
 data = df_mean, linewidth = 1)+
guides(fill = guide_legend(nrow = 2, byrow = TRUE),
 color = guide_legend(nrow = 2, byrow = TRUE))+
scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
coord_cartesian(clip = 'off')+
geom_rug(sides = 'b', aes(x = emissions,
 color = sources),
 data = df, alpha = 0.5, outside = TRUE)
} else {
 ggplot(data = df, aes(emissions, fill = sources))+
 geom_density(alpha = 0.5, bounds = c(0, Inf))+
 ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
 ggtitle(paste0(type, " source test run densities"))+
 multi_source_theme()+labs(fill = 'Top sources',
 color = 'Top sources')+
 geom_vline(aes(xintercept = means, color = sources),
 data = df_mean, linewidth = 1)+
 guides(fill = guide_legend(nrow = 2, byrow = TRUE),
 color = guide_legend(nrow = 2, byrow = TRUE))+
 scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
 scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
 coord_cartesian(clip = 'off')+
 geom_rug(sides = 'b', aes(x = emissions,
 color = sources),
 data = df, alpha = 0.5, outside = TRUE)
}
...

```

Analysis\_results.Rmd

```

`r type` Source `r HAP` `r subcat_text`

```{r, eval = UPL_warn, echo = FALSE, results = 'asis'}
cat(paste0("The ", HAP, " ", subcat_text, " UPL calculated for the New source data set, ",
  ↪ signif(UPL_old, 3), " was larger than the UPL for the Existing source data set.
  ↪ Therefore, the Existing source UPL was used instead."))
...

The distribution that best represents the `r type` source data set for `r HAP` `r
  ↪ subcat_text` was the `r distr` distribution, with a corresponding UPL of `r
  ↪ signif(results$fit_table$UPL, digits = 4)` (`r meas_unit`). The `r distr`
  ↪ distribution was the best fit distribution with a $$SSE$ of `r
  ↪ signif(results$fit_table$SSE, digits = 4)`, `r results$fit_table$Obs_in_CI` out of `r
  ↪ nrow(df)` emission values within the 95% CI of predictions, and the probability
  ↪ density function integrated to approximately `r
  ↪ signif(results$fit_table$pdf_integral, digits = 2)`.

```{r, echo = FALSE, appendix = FALSE, fig.dim = c(5, 4), fig.align = "center", fig.cap =
 ↪ "Observation density of emissions for population of test runs. The obseration data
 ↪ are indicated in black as points and a rug along the axis, with the observation
 ↪ density distribution as a black line. The fitted distribution that is the basis of
 ↪ the UPL estimate is colored orange. The average of the emission test runs is the
 ↪ vertical black line and the UPL result is the vertical orange line."}

```

```

if (!is.na(subcat_name)){
 ggplot()+
 geom_line(data = results$pred_pdf_dat, size = 0.75,
 aes(y = ydens, x = x_hat, color = 'a'))+
 geom_area(data = results$pred_pdf_dat, alpha = 0.25,
 aes(y = ydens, x = x_hat, fill = 'a'))+
 geom_point(aes(y = ydens, x = emissions), data = results$obs_pdf_dat,
 size = 3, alpha = 0.5, shape = 19, color = 'black')+
 geom_errorbar(data=results$obs_pdf_dat,alpha=0.5,
 aes(ymin=low,ymax=up,x=emissions,color='b'))+
 geom_point(data=results$obs_pdf_dat,
 aes(x=emissions,y=med,fill='b'),
 size=3,alpha=0.5,shape=21,color='black')+
 geom_line(aes(y = pdf_hat, x = x_hat, color = 'b'),
 data = results$pred_pdf_dat, size = 0.75, linetype = 2)+
 geom_area(aes(y = pdf_hat, x = x_hat, fill = 'b'), alpha = 0.25,
 data = results$pred_pdf_dat)+
 ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
 ggtitle(paste0(subcat_name, " ", subcat_level, " ", type,
 " source\nMACT floor test run population"))+
 pop_distr_theme()+
 geom_vline(aes(xintercept = (mean(df$emissions)),
 color = 'a'), linewidth = 1, linetype = 1)+
 geom_vline(aes(xintercept = resultsfit_tableUPL, color = 'b'),
 linewidth = 1, linetype = 2)+
 scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
 scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
 coord_cartesian(clip = 'off')+
 labs(color = 'Distribution:', fill = 'Distribution:')+
 geom_rug(sides = 'b', aes(x = emissions), data = results$obs_pdf_dat,
 alpha = 0.5, outside = TRUE, color = 'black')+
 scale_color_manual(values = c('black', '#FF7F00'),
 labels = c('Observations', distr))+
 scale_fill_manual(values = c('black', '#FF7F00'),
 labels = c('Observations', distr))
} else {
 ggplot()+
 geom_line(data = results$pred_pdf_dat, size = 0.75,
 aes(y = ydens, x = x_hat, color = 'a'))+
 geom_area(data = results$pred_pdf_dat, alpha = 0.25,
 aes(y = ydens, x = x_hat, fill = 'a'))+
 geom_point(aes(y = ydens, x = emissions), data = results$obs_pdf_dat,
 size = 3, alpha = 0.5, shape = 19, color = 'black')+
 geom_errorbar(data=results$obs_pdf_dat,alpha=0.5,
 aes(ymin=low,ymax=up,x=emissions,color='b'))+
 geom_point(data=results$obs_pdf_dat,
 aes(x=emissions,y=med,fill='b'),
 size=3,alpha=0.5,shape=21,color='black')+
 geom_line(aes(y = pdf_hat, x = x_hat, color = 'b'),
 data = results$pred_pdf_dat, size = 0.75, linetype = 2)+
 geom_area(aes(y = pdf_hat, x = x_hat, fill = 'b'), alpha = 0.25,
 data = results$pred_pdf_dat)+
 ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
 ggtitle(paste0(type,

```

```

 " source\nMACT floor test run population")))+
pop_distr_theme()+
geom_vline(aes(xintercept = (mean(df$emissions)),
 color = 'a'), linewidth = 1, linetype = 1)+
geom_vline(aes(xintercept = resultsfit_tableUPL, color = 'b'),
 linewidth = 1, linetype = 2)+
scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
coord_cartesian(clip = 'off')+
labs(color = 'Distribution:', fill = 'Distribution:')+
geom_rug(sides = 'b', aes(x = emissions), data = results$obs_pdf_dat,
 alpha = 0.5, outside = TRUE, color = 'black')+
scale_color_manual(values = c('black', '#FF7F00'),
 labels = c('Observations', distr))+
scale_fill_manual(values = c('black', '#FF7F00'),
 labels = c('Observations', distr))
}
if (is.null(bw)){
 bw_result = sd(df$emissions) * nrow(df)^(-2/5)
} else if (is.character(bw)){
 raw_dens = npuniden.boundary(X = df$emissions, Y = xvals,
 a = low, b = up, proper = TRUE,
 kertype = kernel_arg, bwmethod=bw)

 bw_result = raw_dens$h
} else if (is.numeric(bw)){
 bw_result = bw
}
if (any(results$conv_output$convYN != 'Yes')){
 conv_text = TRUE
} else {
 conv_text = FALSE
}
...

```

The density plot above of emission test run data was created using `ggplot2` in `R`. The  
 ↳ density curves were generated using `obs_density()` in `UPLforOAR` version `R`  
 ↳ `packageVersion('UPLforOAR')`, using a `R kernel_arg` kernel type, bounds of (`R`  
 ↳ `c(low, up)`), and a bandwidth of `R signif(bw_result, 3)`. For further details see  
 ↳ the `R` code in Appendix B.

```

```{r, eval = !conv_text, echo = FALSE, results = 'asis'}
cat(paste0("The ", distr, " distribution's parameters converged well, passing all
↳ Gelman-Rubin diagnostic checks and with well-mixed MCMC chains (left figures below)
↳ and an approximately normal posterior distributions around the converged value (right
↳ figures below)."))
...

```{r, eval = conv_text, echo = FALSE, results = 'asis'}
cat(paste0("The ", distr, " distribution's parameters did not converge, failing some
↳ Gelman-Rubin diagnostic checks. Check the figures below to see if MCMC chains (left
↳ figures below) are well-mixed and posterior distributions are approximately normal
↳ around the converged value (right figures below). Results from this distribution
↳ model should not be used until the convergence issues is corrected. Consider a more
↳ appropriate choice of distribution, or manually specifying better priors and
↳ likelihood limits."))

```

```

...
```{r, echo = FALSE, results = FALSE, appendix = TRUE, fig.height = 4, out.width =
↵ "100%", fig.align = "center", fig.cap = "Convergence figures for distribution
↵ parameters."}
part1 = setup_likelihood(distribution = distr, data = df,
                        emissions = 'emissions', RNG.state = RNG.state,
                        manual_prior = manual_prior, random = FALSE,
                        prior_list = prior_list)
part2 = run_likelihood(model_input = part1, future_runs = future_runs,
                      xvals = xvals, maxY = maxY, minY = minY)
part3 = converge_figs(distribution = distr, jags_model_run = part2)
part3
...

```

Appendix_data.Rmd

```

`r type` source MACT floor analysis data set for HAP `r HAP` `r subcat_text`:

```{r, echo = FALSE, appendix = FALSE}
col_names = c(emissions, sources, 'log emissions')
df %>% knitr::kable(
 caption = "Top performing source emissions data used in MACT floor
 UPL calculations.", digits = 12, col.names = col_names)
...

```